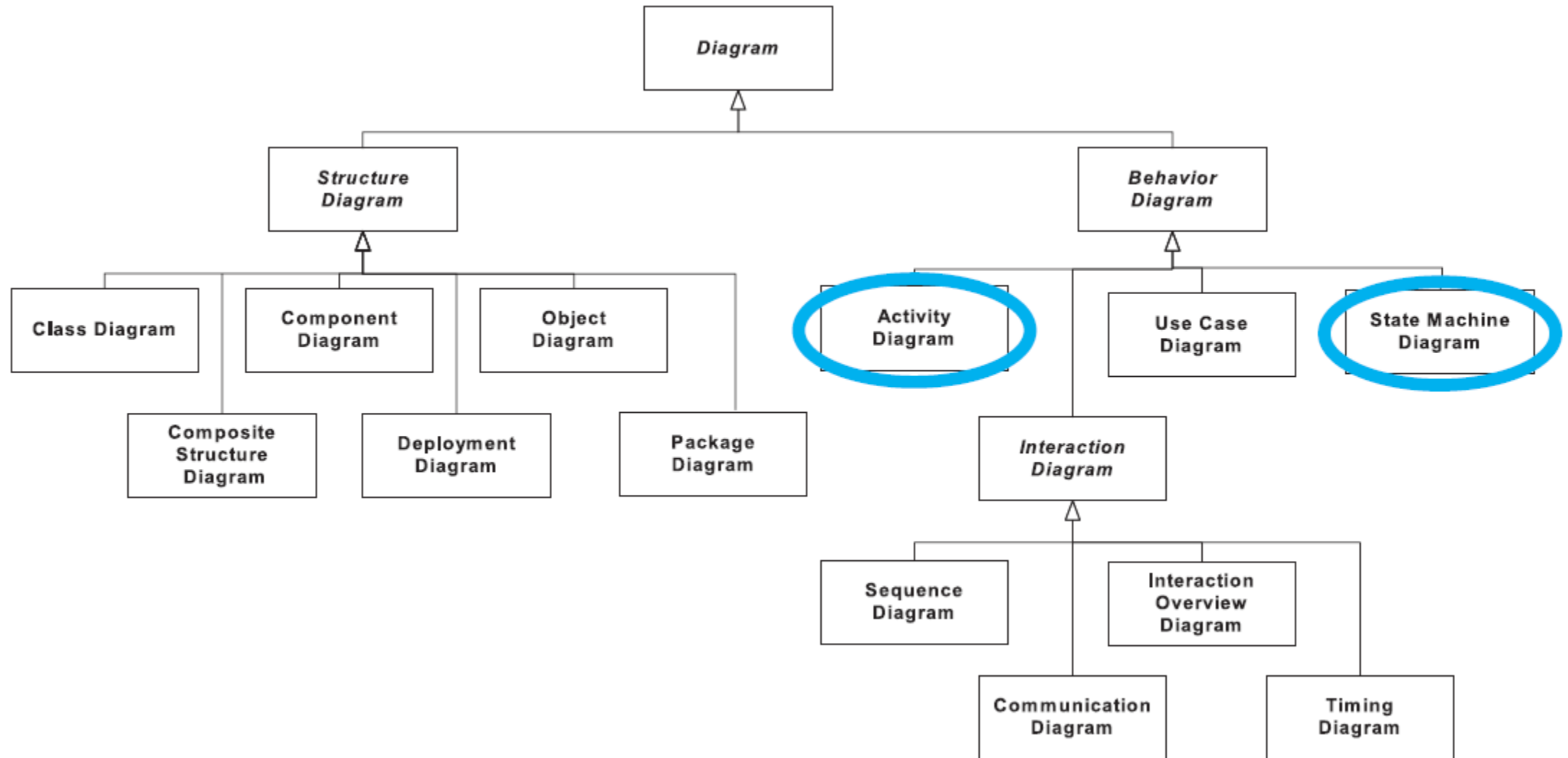


**UNIFIED
MODELING
LANGUAGE™**



TOPIC 1

I diagrammi di attività e stato



Diagrammi di attività e stato: Cosa sono e a cosa servono

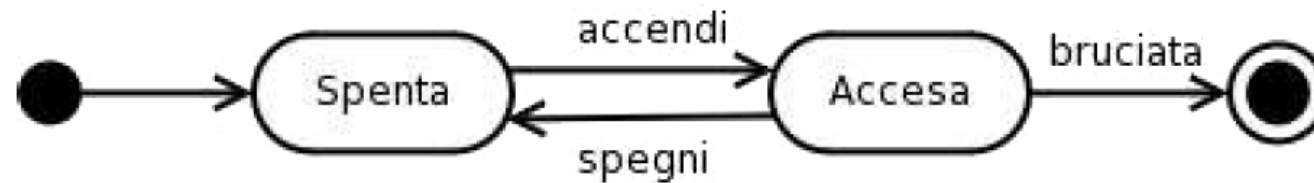
- I diagrammi di attività (activity diagram) e stato (state machine diagram) sono diagrammi che descrivono comportamento.
- Il diagramma di attività modella un comportamento (che riguarda una o più entità) come un insieme di azioni organizzate secondo un flusso.
- Il diagramma di stato modella il comportamento (generalmente di una sola entità) come variazioni del suo stato interno.

TOPIC 2

I diagrammi di stato

Diagramma di Stato

- Sono diagrammi di flusso orientati agli oggetti.
- Consentono di modellare un processo tramite
 - Stati
 - Transizioni tra stati
- Usati per modellare il ciclo di vita di una entità reattiva



- Descrive la macchina a stati di una singola entità
- Modella il comportamento dinamico di:
 - Classi
 - Casi d'uso
 - Sottosistemi
 - Sistemi interi

- Stato



- Stato iniziale



- Stato finale



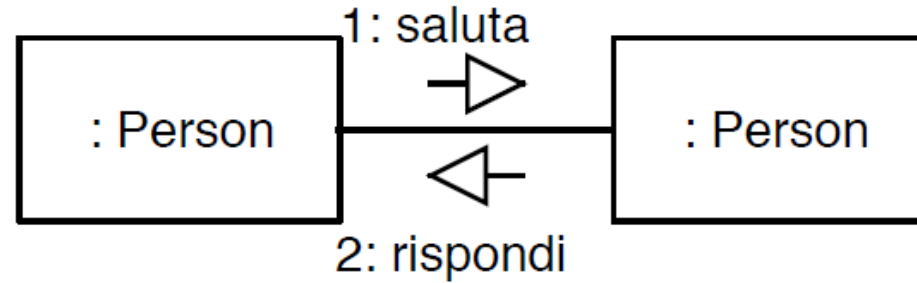
- Transizioni



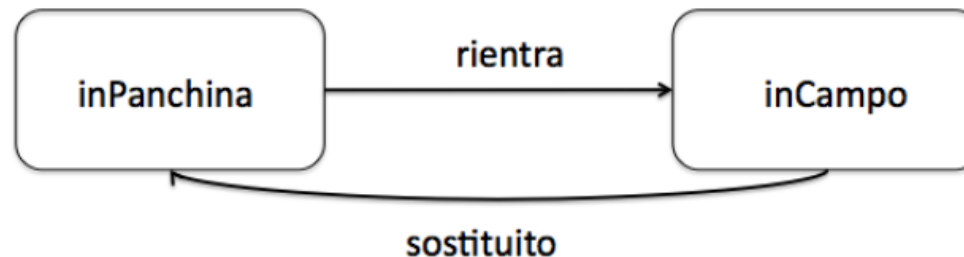
- Gli eventi sono istantanei. Il nome dell'evento è scritto sulla freccia della transizione.

Un passo indietro: Interazione vs. Macchina a stati

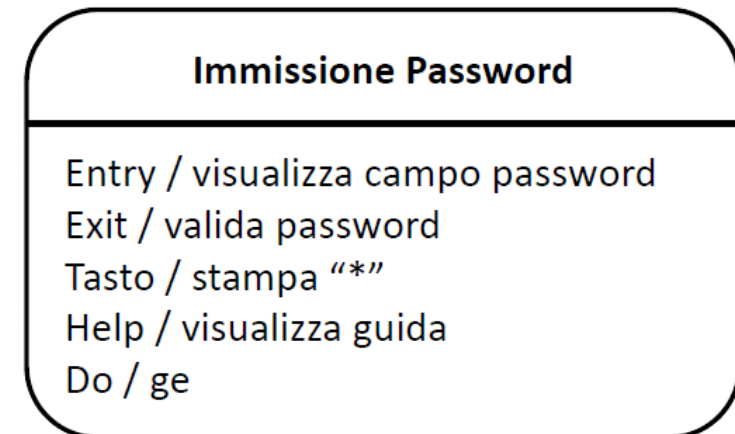
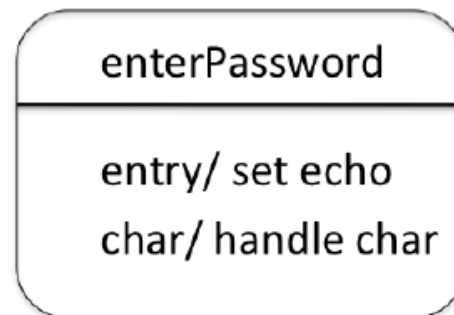
- Interazione: un insieme di oggetti che si scambiano messaggi per raggiungere un dato obiettivo



- Macchina a stati: descrive la sequenza di stati in cui si trova un oggetto durante il suo ciclo di vita e in risposta a eventi

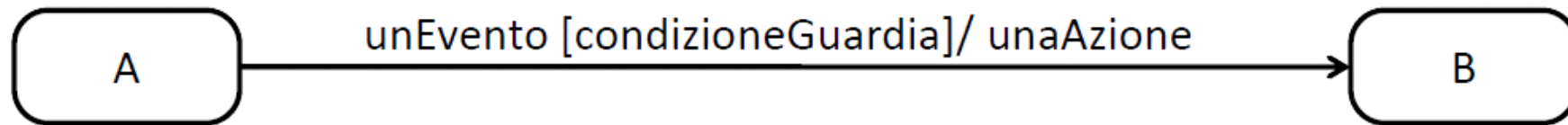


- Qualunque classificatore UML può essere associato a una macchina a stati che descrive il funzionamento delle sue istanze.
- Uno stato è una condizione o situazione nella vita di un oggetto in cui esso:
 - Soddisfa una condizione
 - Esegue un'attività o
 - aspetta un evento



- La macchina a stati riceve occorrenze di eventi che vengono salvati su una coda ed estratti uno alla volta.
- La semantica di questi eventi è di tipo run-to-completion: l'occorrenza di un evento viene estratta solo dopo che la macchina a stati ha finito di processare quella precedente.
- Se ci sono più transizioni eseguibili in un dato momento (es. 2 transizioni dallo stesso stato con lo stesso evento e due condizioni diverse, entrambe vere), solo una viene eseguita.

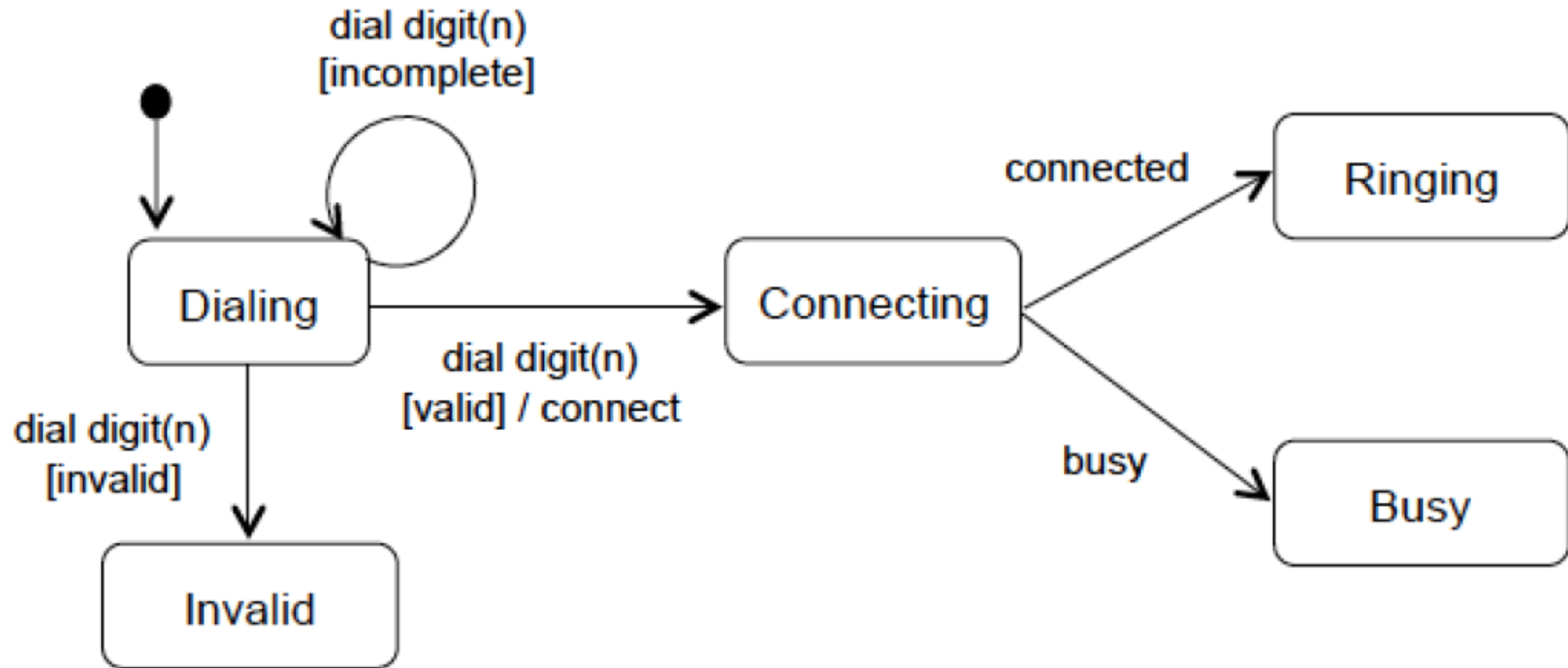
- Quando avviene «unEvento», se la «condizioneGuardia» risulta vera, allora esegue «unaAzione», quindi entra immediatamente nello stato B



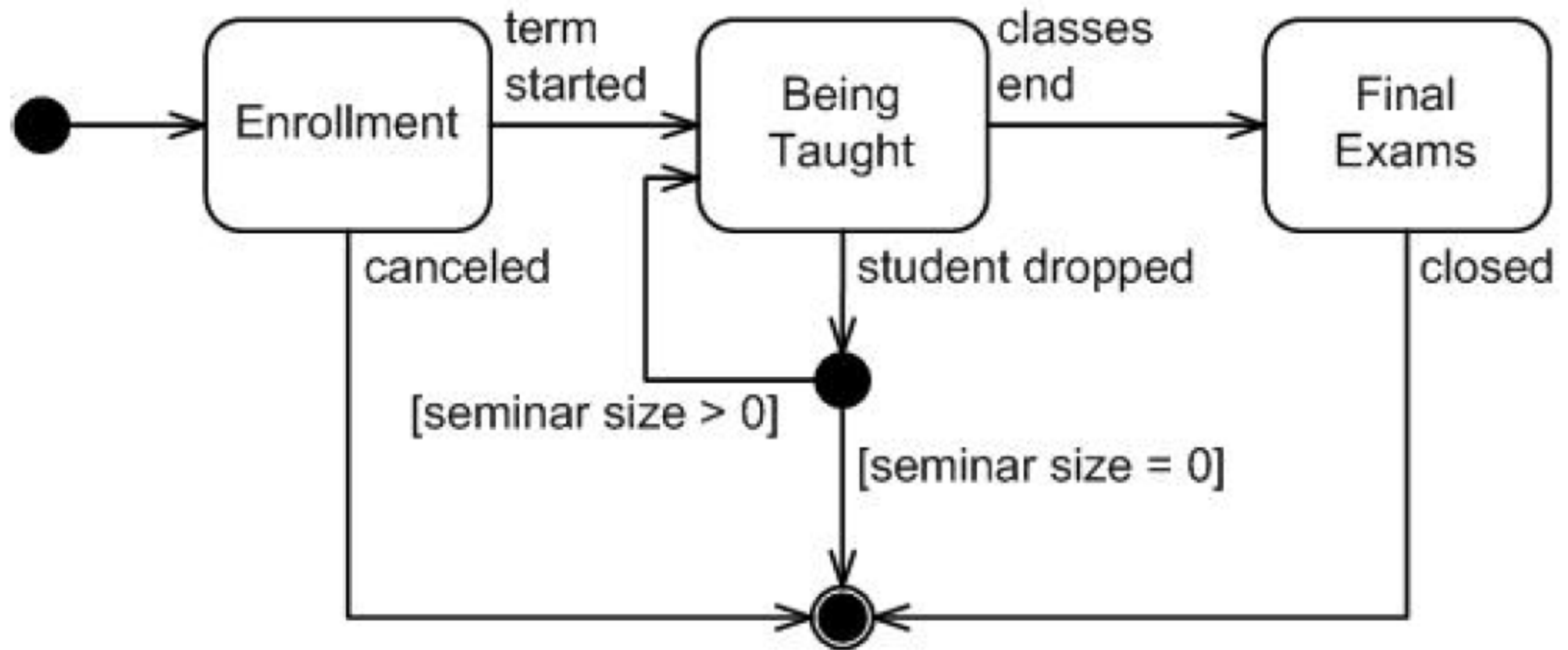
- Ogni transizione, oltre allo stato origine e destinazione, può specificare:
 - Event: un 'trigger' che attiva il passaggio di stato
 - Guard: una condizione che, se vera, permette il passaggio di stato
 - Action: un'azione che risulta dal cambio di stato
- Sintassi: event[guard]/action
- La transizione avviene come risposta a uno degli eventi (quando la guardia è vera), e al momento della transizione il contesto esegue l'azione specificata
- Sono tutti opzionali

- Un evento è la specifica di un'occorrenza di interesse che ha una collocazione nel tempo e nello spazio.
 - Evento di chiamata
 - Richiesta di esecuzione di una serie di azioni
 - Evento di segnale
 - Ricezione di un segnale
 - Evento di variazione
 - Se si verificano particolari condizioni
 - Evento temporale
 - In momenti determinati

Un esempio completo

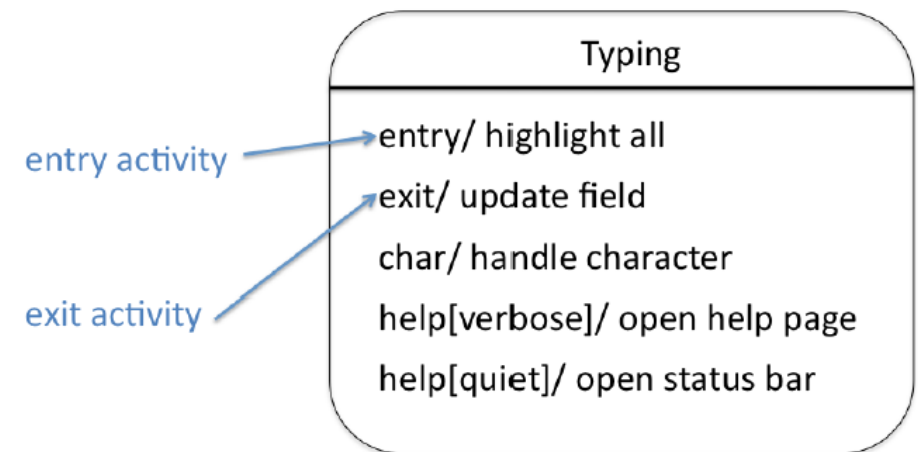


Un esempio completo



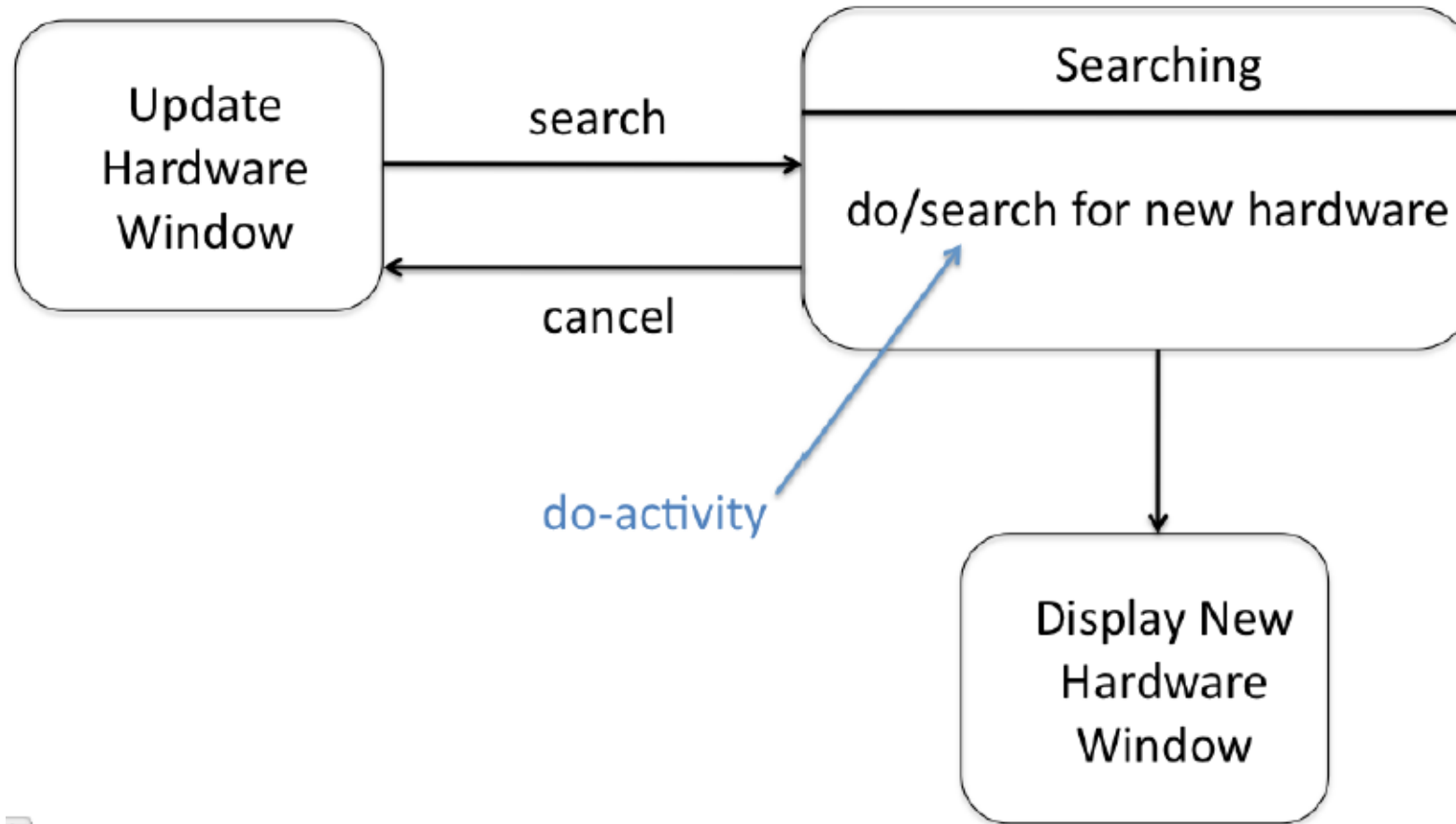
- Uno stato può attivare delle azioni
 - Entry: Attivata all'ingresso nello stato
 - Do: Si svolge nel ciclo di vita dello stato
 - Event: Attivata in risposta ad un evento
 - Exit: Attivata prima dell'uscita dallo stato
 - Include: Chiama una macchina a stati inclusa (submachine)

- Uno stato può reagire ad eventi (e verificare condizioni) anche senza una transizione ad uno stato diverso
- Le internal activities sono mostrate nel secondo slot e seguono la stessa sintassi delle transizioni
- Simile ad una self-transition
- Esempio: riempimento di un campo di testo in un form



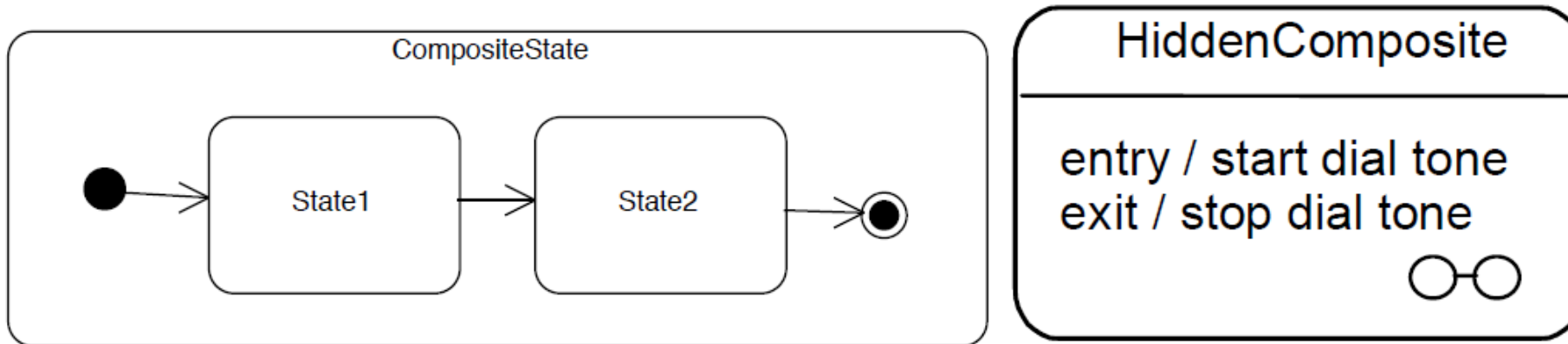
- All'interno di uno stato si possono usare alcune azioni speciali, indicate tramite keyword:
 - entry: eseguita quando l'oggetto entra nello stato
 - exit: eseguita quando l'oggetto esce dallo stato
 - do (do-activity): eseguita mentre l'oggetto è nello stato
- Una self-transition attiva sempre le entry ed exit, le internal activities invece no
- Una do-activity non è 'istantanea' ma può durare per un intervallo di tempo ed essere interrotta (da altri eventi)

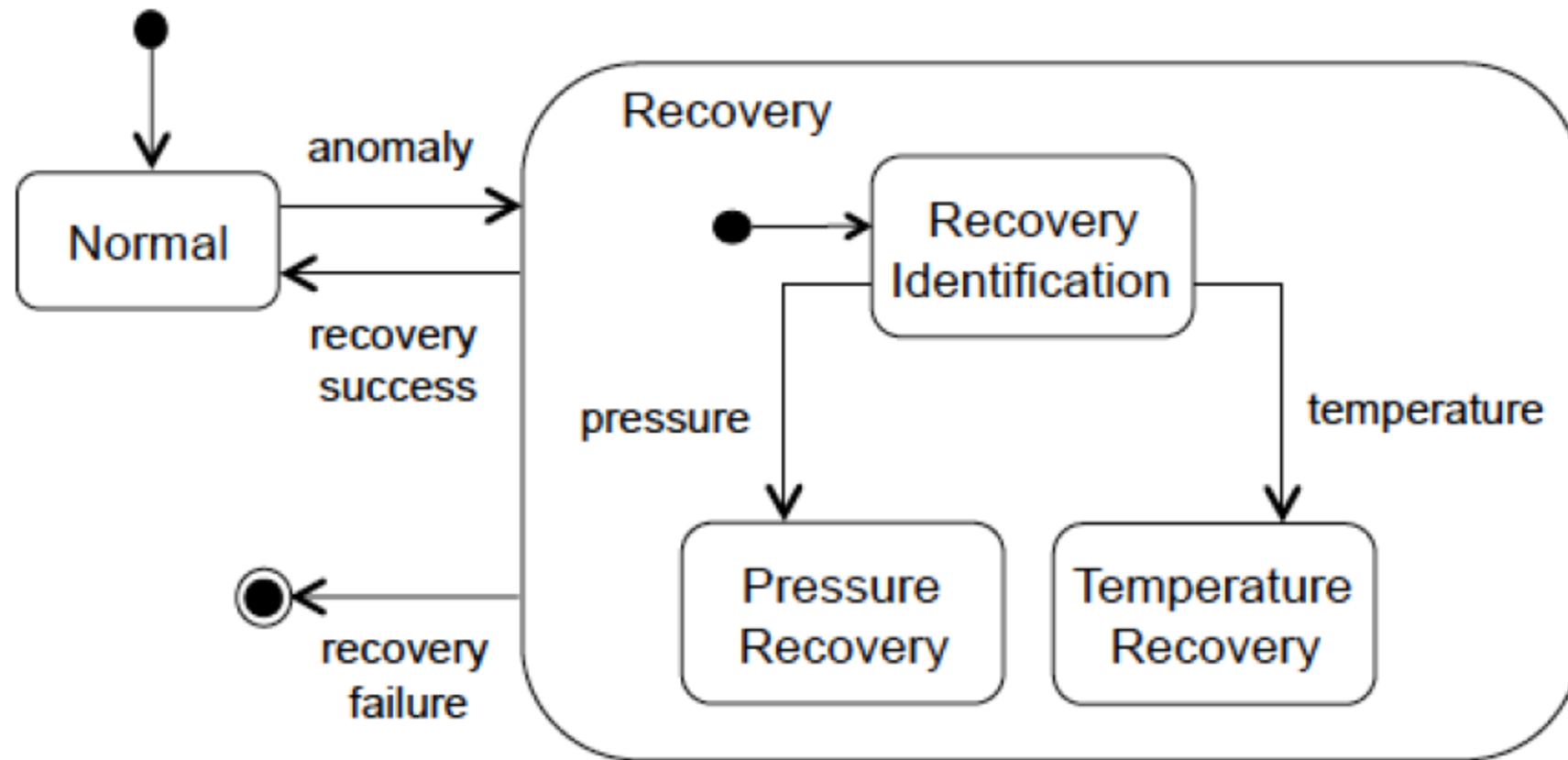
- Modelliamo la ricerca di nuovo hardware da installare su un sistema operativo.



Stati composti

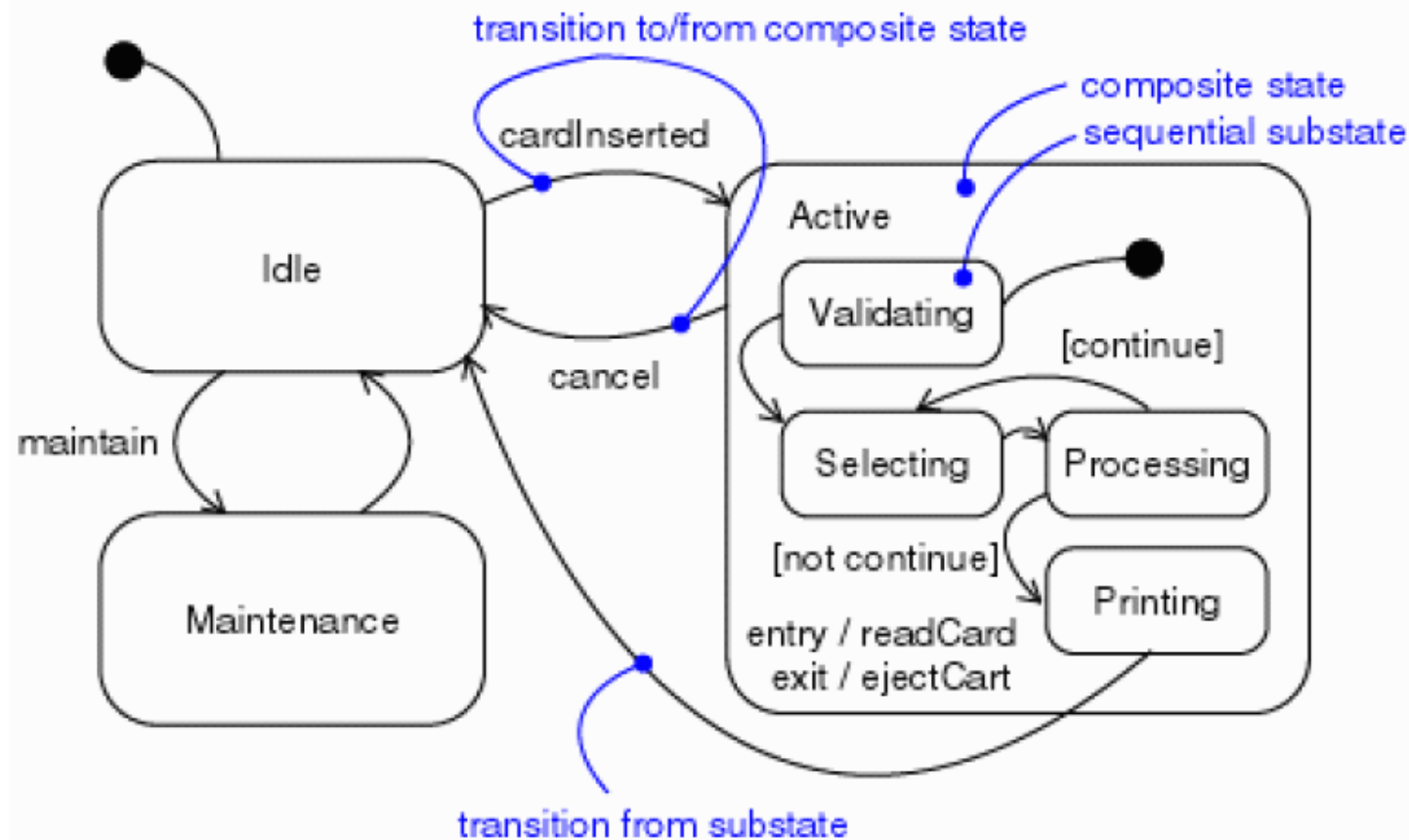
- Permettono di suddividere la complessità del modello: dall'esterno si vede un macro-stato, al cui interno vi sono altri stati.
- Si può anche creare uno stato che fa riferimento ad un altro diagramma di macchina a stati (submachine state).
- Si può usare un'icona per rappresentare uno stato composto il cui comportamento interno non è mostrato.





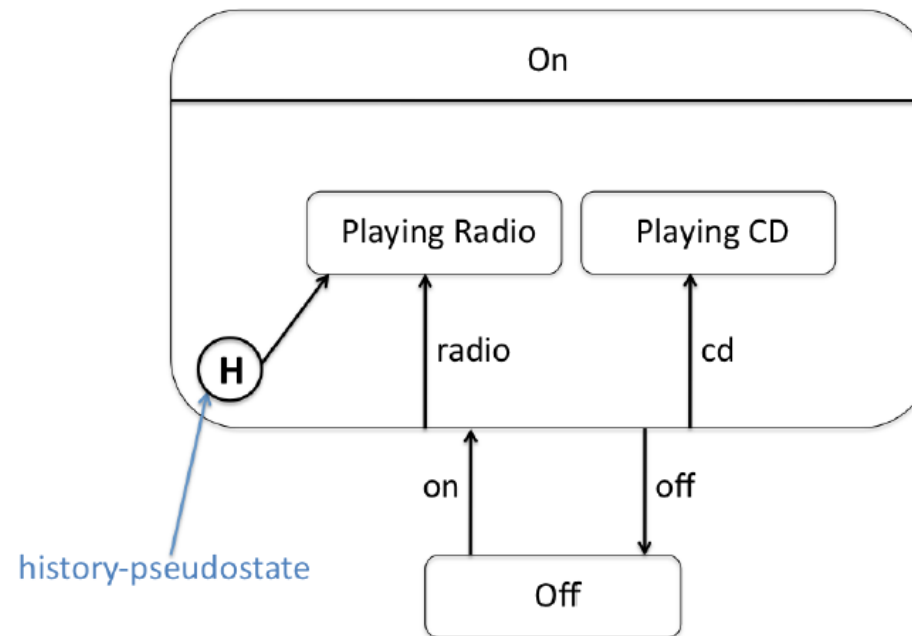
Stati compositi e transizioni

- Si possono definire transizioni da uno stato interno verso stati esterni
- Transizioni in uscita dal bordo dello stato composito e legate ad un evento sono ereditate da tutti gli stati all'interno.

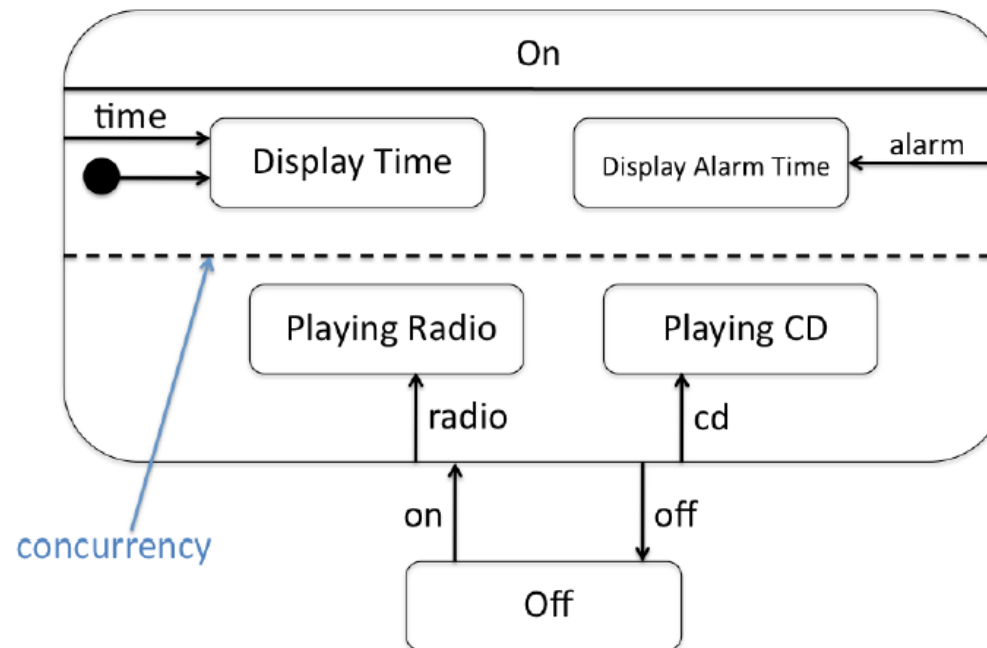


History pseudostate

- Un history pseudostate indica il più recente stato interno attivo di uno stato composito
- Utile per ripristinare stati precedenti
- Esempio: alla ri-accensione di una radio/CD si seleziona radio o CD in base alla scelta attiva allo spegnimento

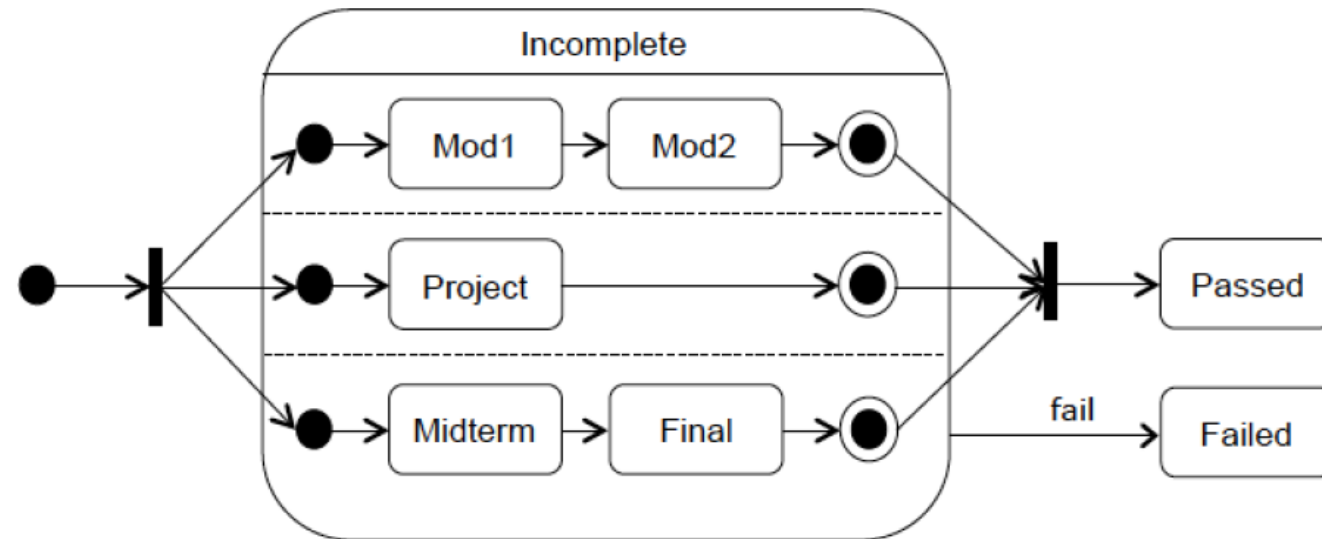


- Gli stati composti sono utili per modellare la concorrenza. Si divide lo stato composto in (sotto-)diagrammi ortogonali eseguiti in mutua esclusione
- Il diagramma sarebbe molto meno chiaro senza questo approccio (bisogna considerare tutte le possibilità)
- Esempio: una radiosveglia che o mostra l'orario o fa ascoltare musica



Stati composti e sincronizzazione

- Gli stati composti sono inoltre utili per modellare la sincronizzazione. Si divide lo stato composto in (sotto-)diagrammi e si usano gli operatori di fork e join
- Esempio: le attività e prove che uno studente deve superare per concludere un corso

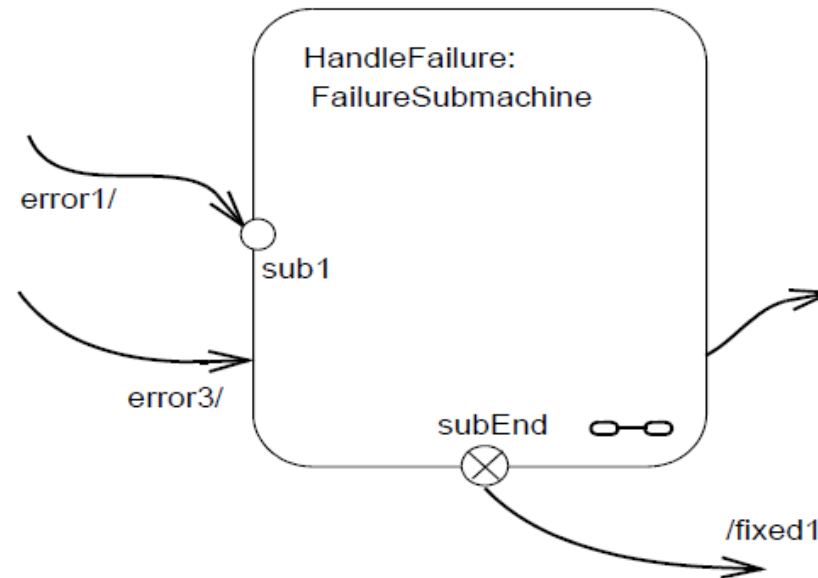


Transizioni di completamento

- Si tratta di transizioni che non hanno un evento associato (ma possono avere una guardia).
- Nel caso di uno stato semplice, una transizione di completamento è eseguita al termine dell'attività di quello stato (fine azioni do).
- Nel caso di uno stato composito o submachine state una transizione di completamento è eseguita quando si giunge in uno stato finale oppure un exit point.

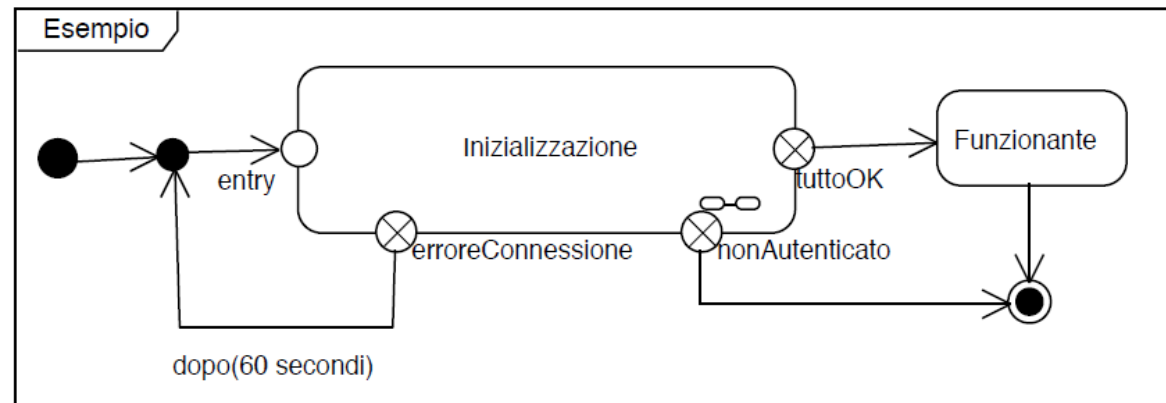
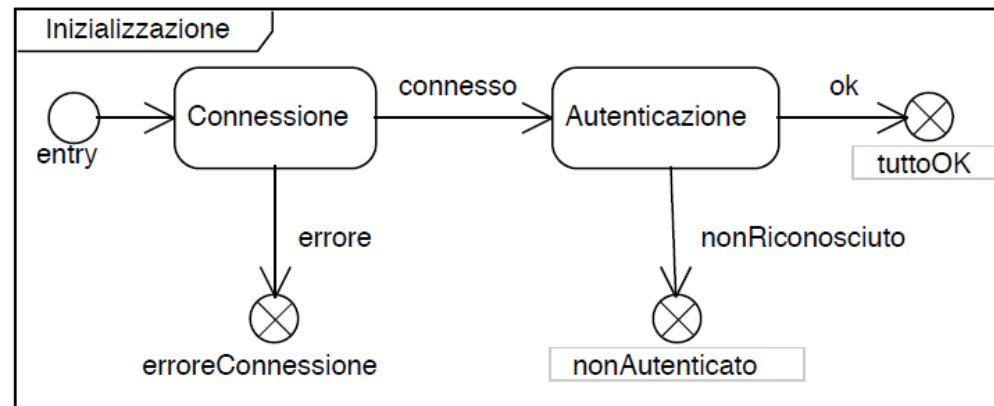
Completamento ed entry/exit points

- L'evento error3 fa partire l'esecuzione dallo stato iniziale dello stato composito.
- L'evento error1 fa partire l'esecuzione dall'entry point sub1.
- Se l'esecuzione termina nell'exit point subEnd si esegue la transizione di completamento che genera il comportamento fixed1.
- Se l'esecuzione termina nello stato finale si segue la transizione sulla destra.

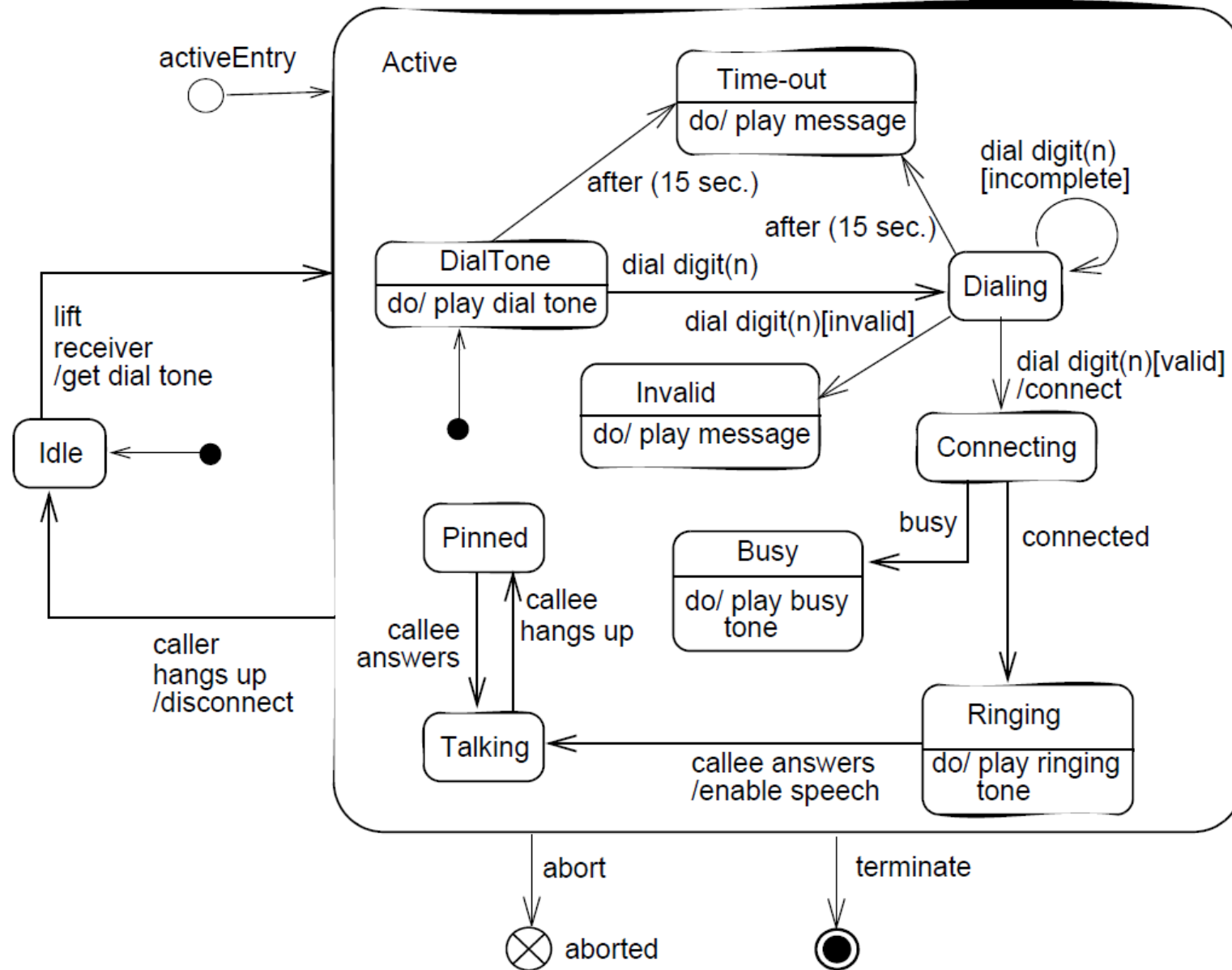


Entry/exit points

- Forniscono un modo per entrare ed uscire dalle macchine a stati in diversi punti (simili a entry/exit points interni)
- Si possono usare in combinazione con i nodi iniziali e finali.



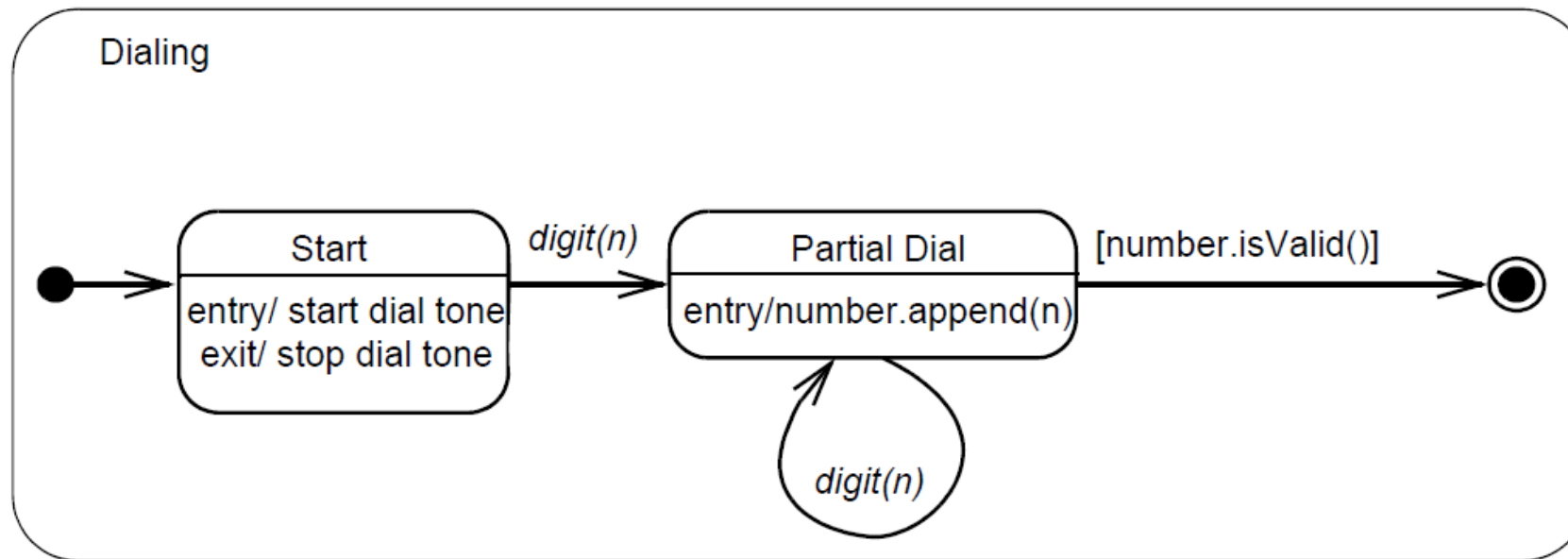
Un esempio completo



- Una macchina a stati può essere o del comportamento o di protocollo (la distinzione è sottile).
- Una macchina a stati del comportamento specifica un comportamento del contesto che va modellato: il modo in cui reagisce agli eventi.
- Una macchina a stati di protocollo specifica quali operazioni possono essere eseguite sul contesto in ogni momento, e può essere associata ad oggetti senza comportamento e perfino non istanziabili (es. interfacce).
- Un classificatore può avere al massimo una m.s. di comportamento e un qualunque numero di m.s. di protocollo.

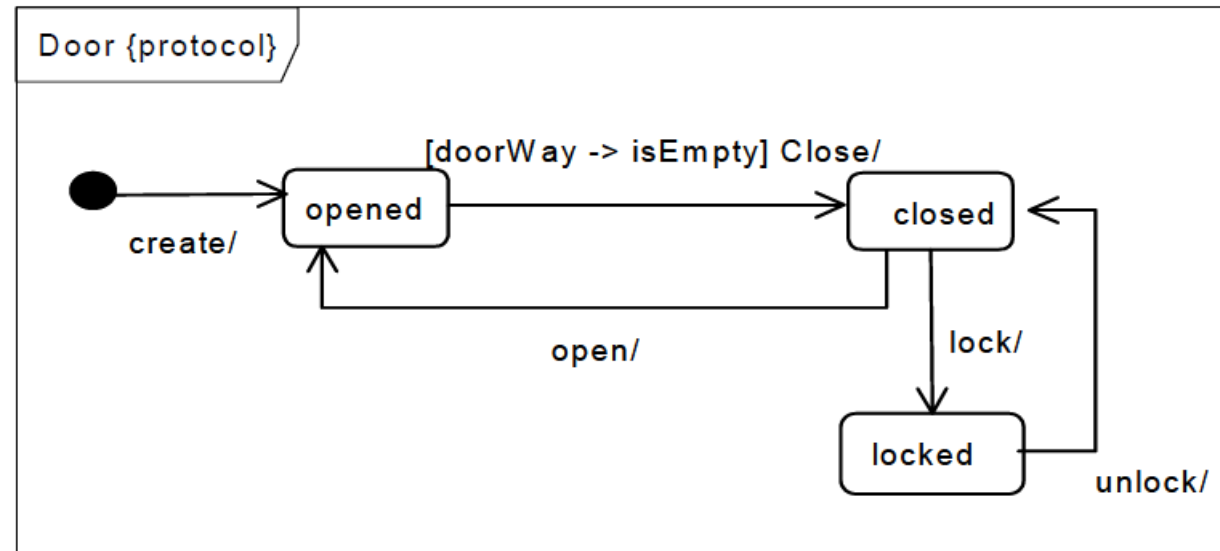
Esempio: m.s. di comportamento

- In una m.s. di comportamento, gli stati possono contenere azioni intraprese all'entrata (entry), all'interno (do) e all'uscita (exit) dello stato.
- Finora abbiamo visto soprattutto questo tipo di diagramma, che è il più comune



Esempio: m.s. di protocollo

- In una m.s. di protocollo, gli eventi nelle transizioni sono generalmente operazioni del classificatore di contesto (in questo caso Door)
- Il diagramma mostra le operazioni legali in ogni momento del suo ciclo di vita.
- L'ordine è particolarmente importante



TOPIC 3

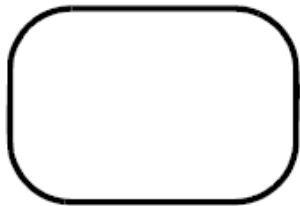
I diagrammi di attività

Il diagramma di attività

- Modella un'attività relativa ad un qualsiasi oggetto, ad esempio:
 - classi
 - casi d'uso
 - interfacce
 - componenti
 - interfacce
 - operazioni di classe
- Alcuni usi dei diagrammi di attività:
 - modellare il flusso di un caso d'uso (analisi)
 - modellare il funzionamento di un'operazione di classe (progettazione)
 - modellare un algoritmo (progettazione)

Attività: ingredienti

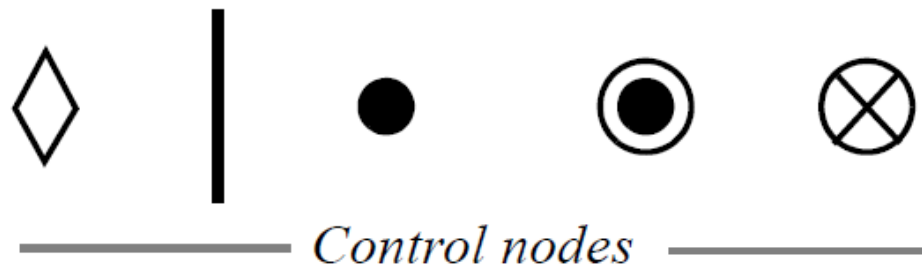
- Nodi azione: specificano unità di comportamento.
- Nodi oggetto: specificano oggetti usati come input e output di azioni.
- Nodi controllo: specificano il flusso dell'attività



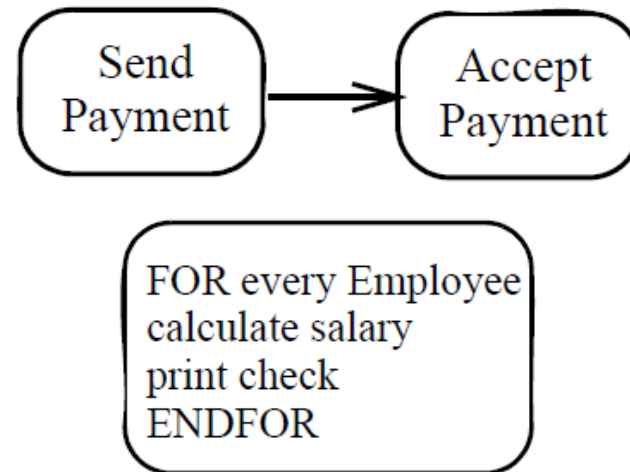
Action node



Object node

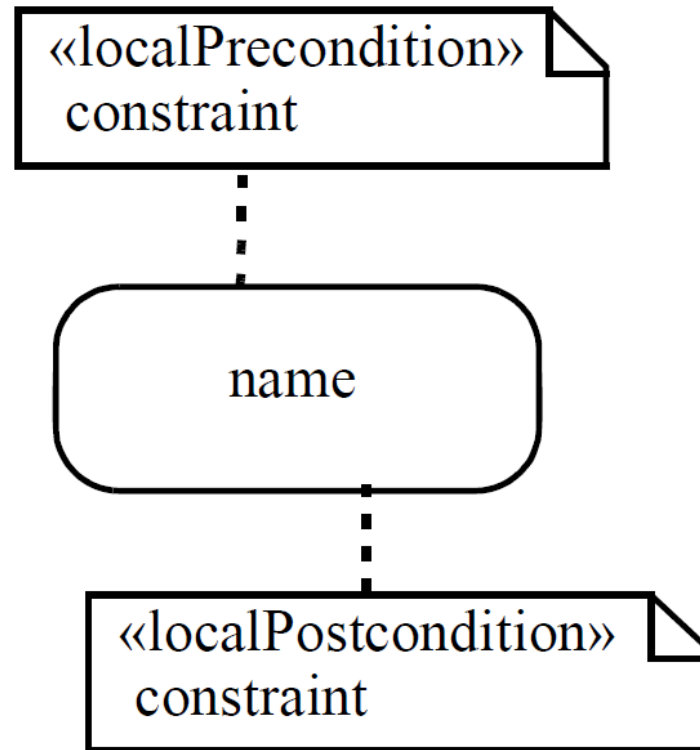


- Un'azione può invocare un'attività, un comportamento o un'operazione.
- Come gli altri elementi di UML, anche le azioni accettano livelli di dettaglio e linguaggi differenti
- Al contrario dei messaggi nei diagrammi di interazione, le azioni non costringono il modellatore a definire tutte le entità in gioco
- Le transizioni (frecce) tra azioni possono avere una guardia

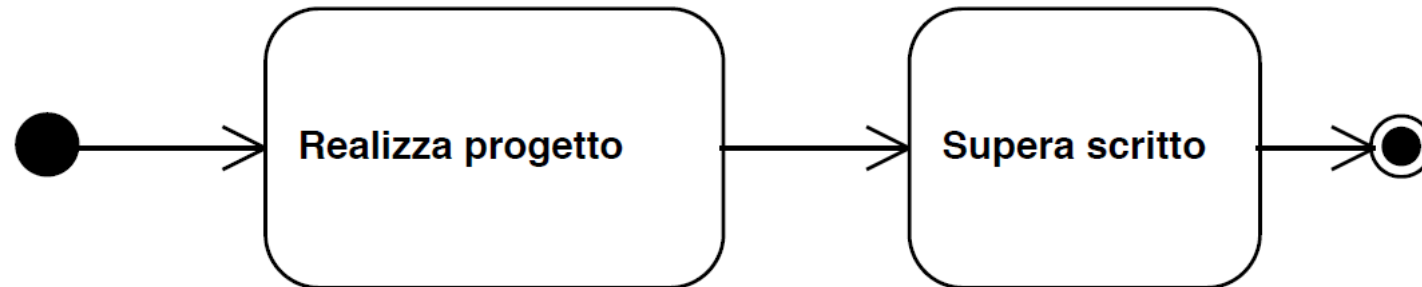


- Per capire la semantica dei diagrammi di attività, bisogna immaginare delle entità, dette token, che viaggiano lungo il diagramma.
- Il flusso dei token definisce il flusso dell'attività.
- I token possono rimanere fermi in un nodo azione/oggetto in attesa che si avveri una condizione su una freccia, oppure una preconditione o postcondizione su un nodo.
- Il movimento di un token è atomico.
- Un nodo azione viene eseguito quando sono presenti token su tutti gli archi in entrata, e tutte le preconditioni sono soddisfatte.
- Al termine di un'azione, sono generati control token su tutti gli archi in uscita.

- Si tratta di condizioni, espresse in qualunque modo, che devono essere soddisfatte per far iniziare o terminare l'azione (permettere a un token di entrare o uscire).

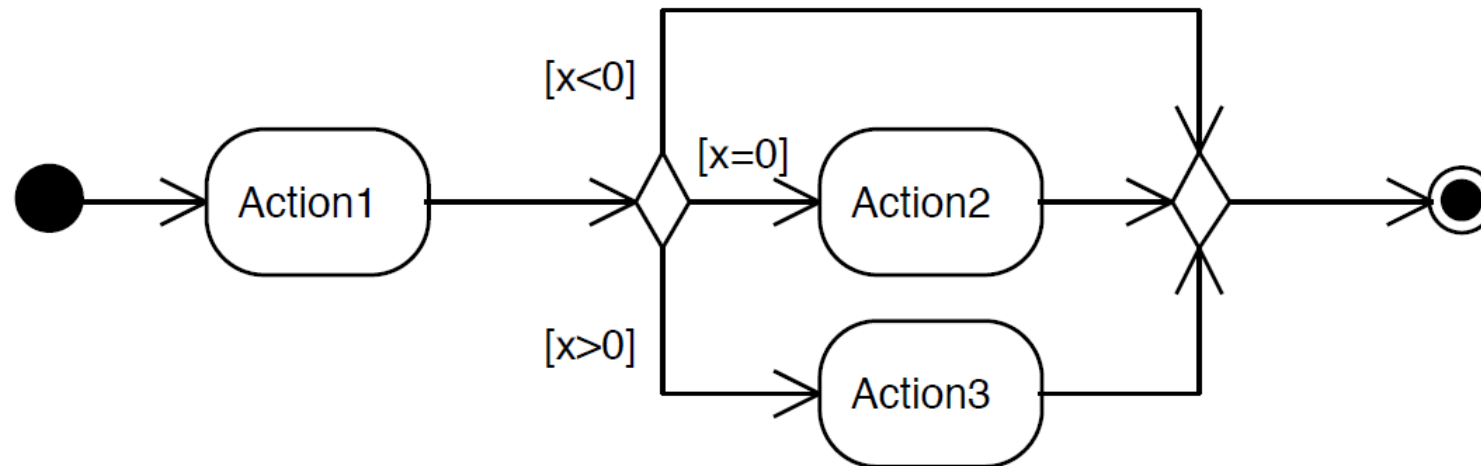


- Il disco nero marca l'inizio dell'attività (genera token).
- Quando un token raggiunge un disco nero bordato (nodo finale), l'attività ha termine.
- Possono comparire in qualunque numero all'interno di un'attività (ogni nodo iniziale fa partire un flusso di esecuzione, il primo nodo finale raggiunto ferma tutti i flussi).

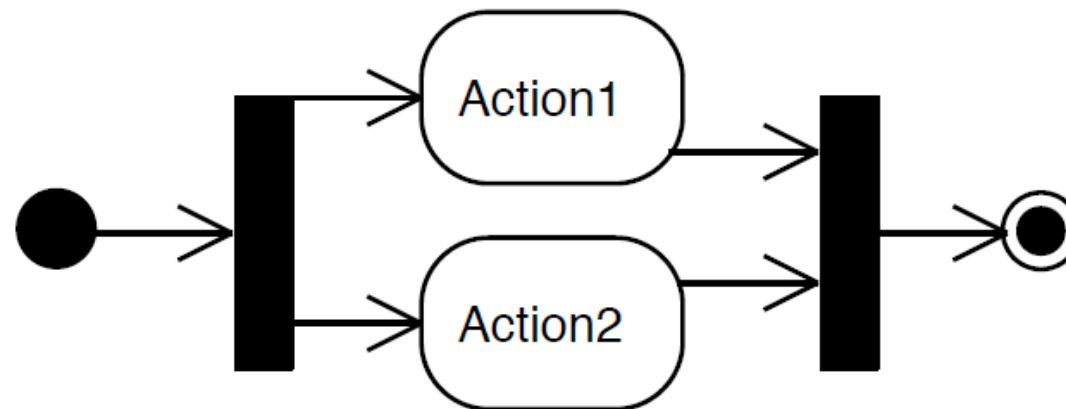


Nodi decisione e fusione

- I nodi decisione hanno un input e vari output mutuamente esclusivi: copiano i token in entrata su uno degli output.
- I nodi fusione hanno vari input e un solo output, sul quale vengono indirizzati tutti i token in ingresso.

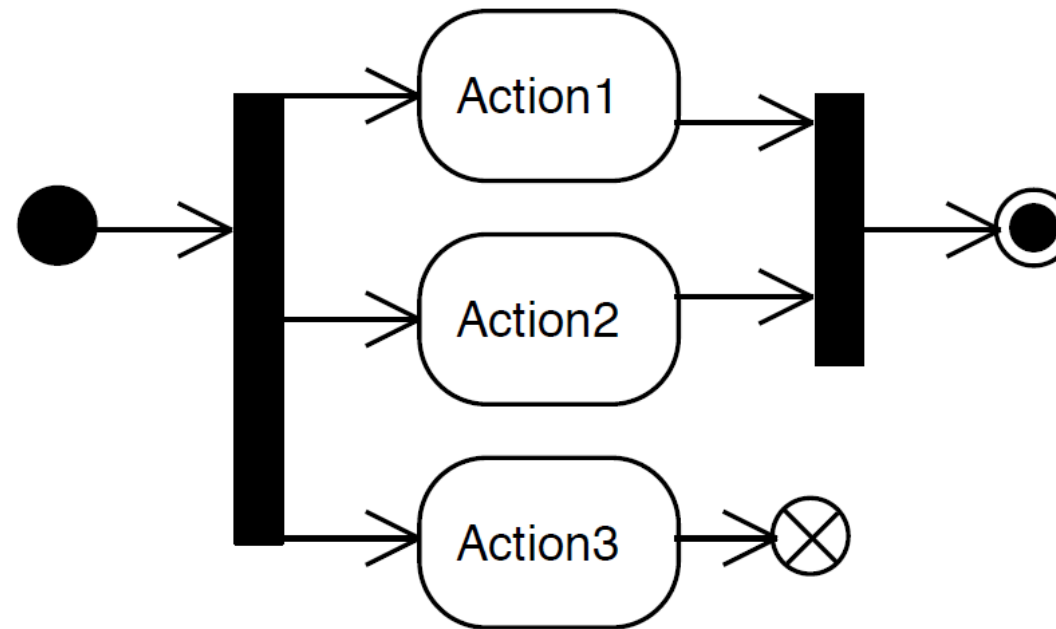


- I nodi fork hanno un ingresso e varie uscite: i token in ingresso sono duplicati su tutte le uscite.
- I nodi join hanno vari ingressi e una sola uscita: quando sono presenti token su tutti gli ingressi, viene prodotto almeno un token in uscita.
- I nodi fork dividono un'esecuzione in più flussi concorrenti, i nodi join sincronizzano e riuniscono i flussi.

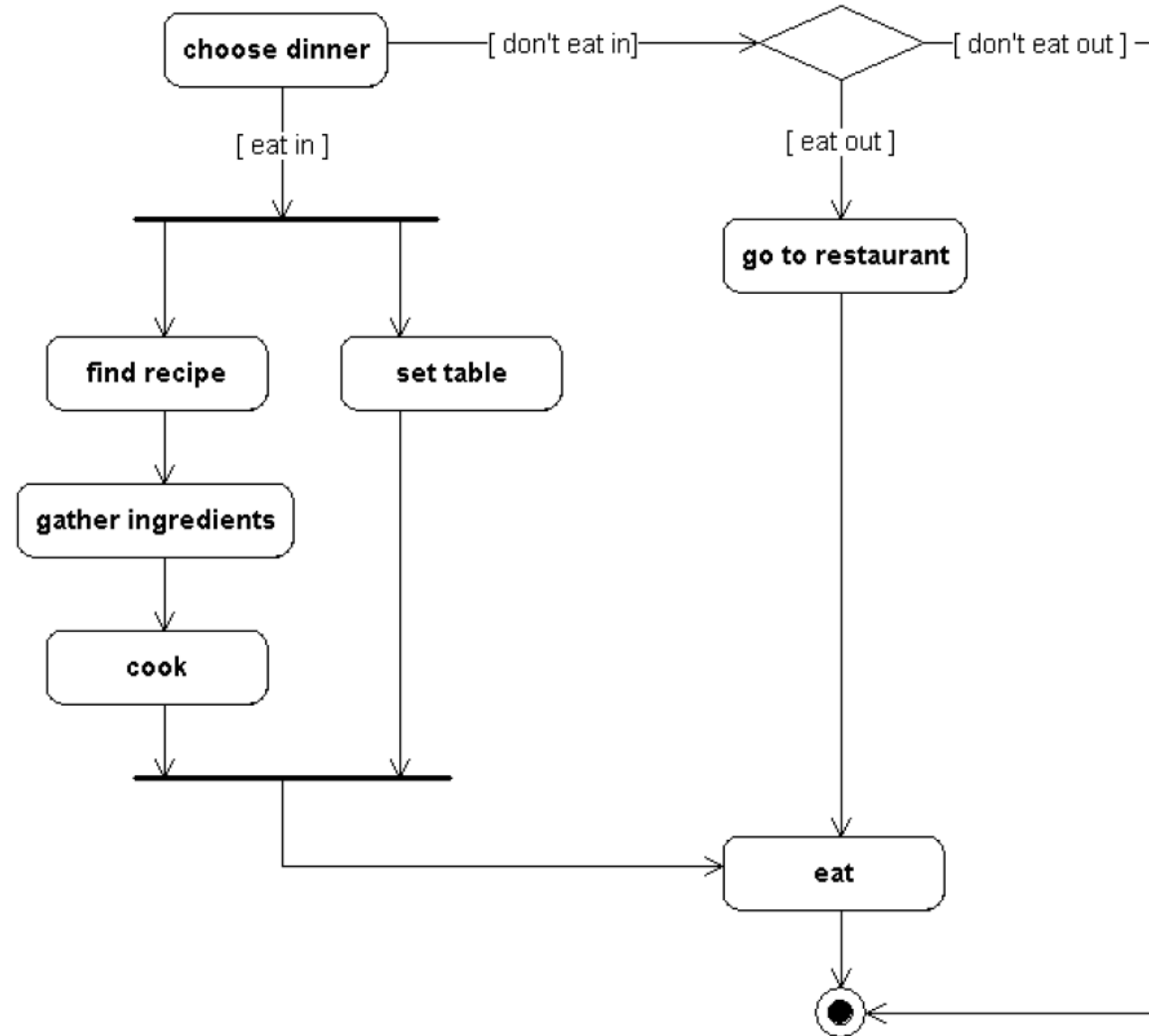


Nodi finali di flusso

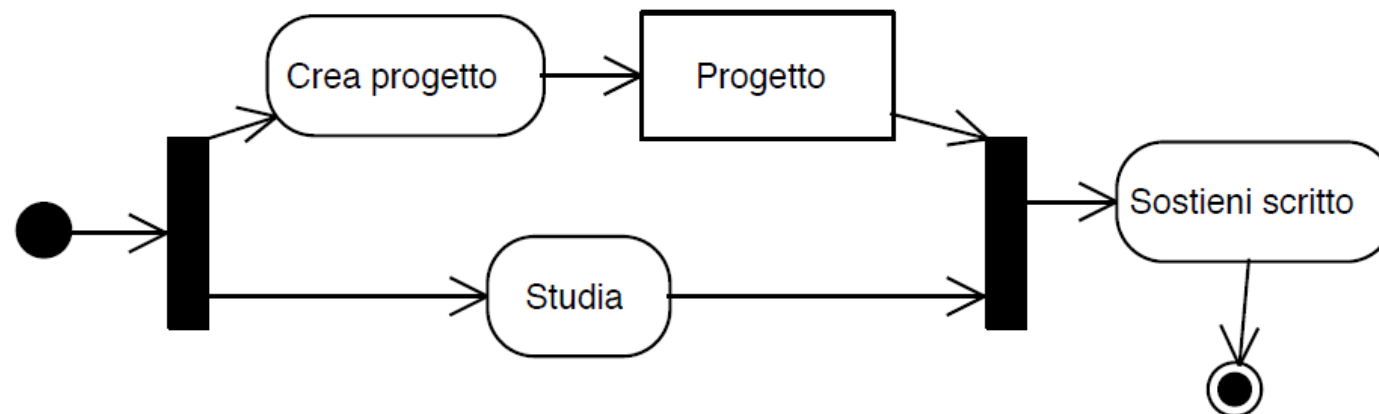
- Quando raggiunti da un token, causano la terminazione solo del flusso che li ha toccati.
- Il raggiungimento di un nodo finale di attività causa comunque la terminazione di tutti i flussi.



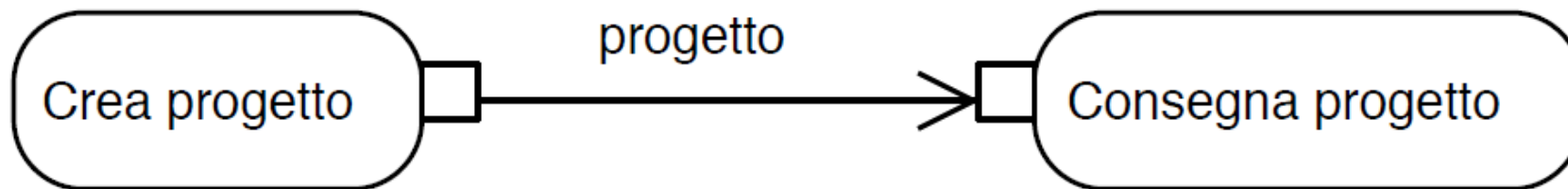
Un esempio completo



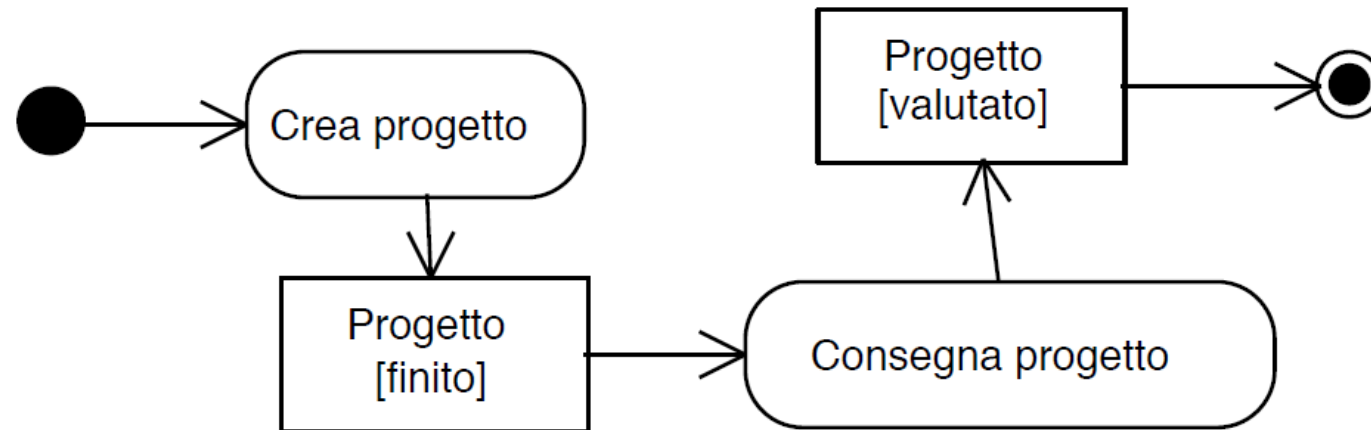
- Servono per modellare gli oggetti in input e output delle azioni
- I token in uscita da questi nodi sono object token, e sono diversi dai control token prodotti dai nodi azione: rappresentano veri e propri oggetti.
- Gli archi in entrata e uscita dai nodi oggetto sono object flow anziché control flow, e ci sono regole che limitano il loro uso (es: gli archi che entrano ed escono dai nodi decisione e fusione devono essere o tutti object o tutti control)



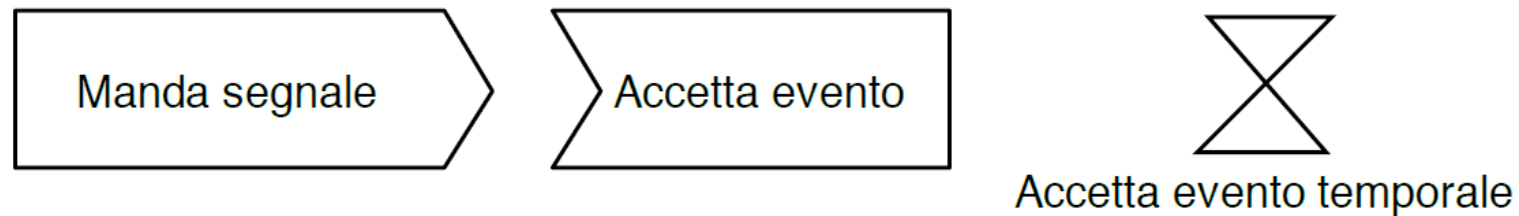
- Si agganciano ai nodi azione per definire un input oppure un output di quell'azione.
- Questa notazione è equivalente a quella di un nodo oggetto tra i due nodi azione.
- I pin aiutano a mostrare i parametri e valori di ritorno di un'azione.



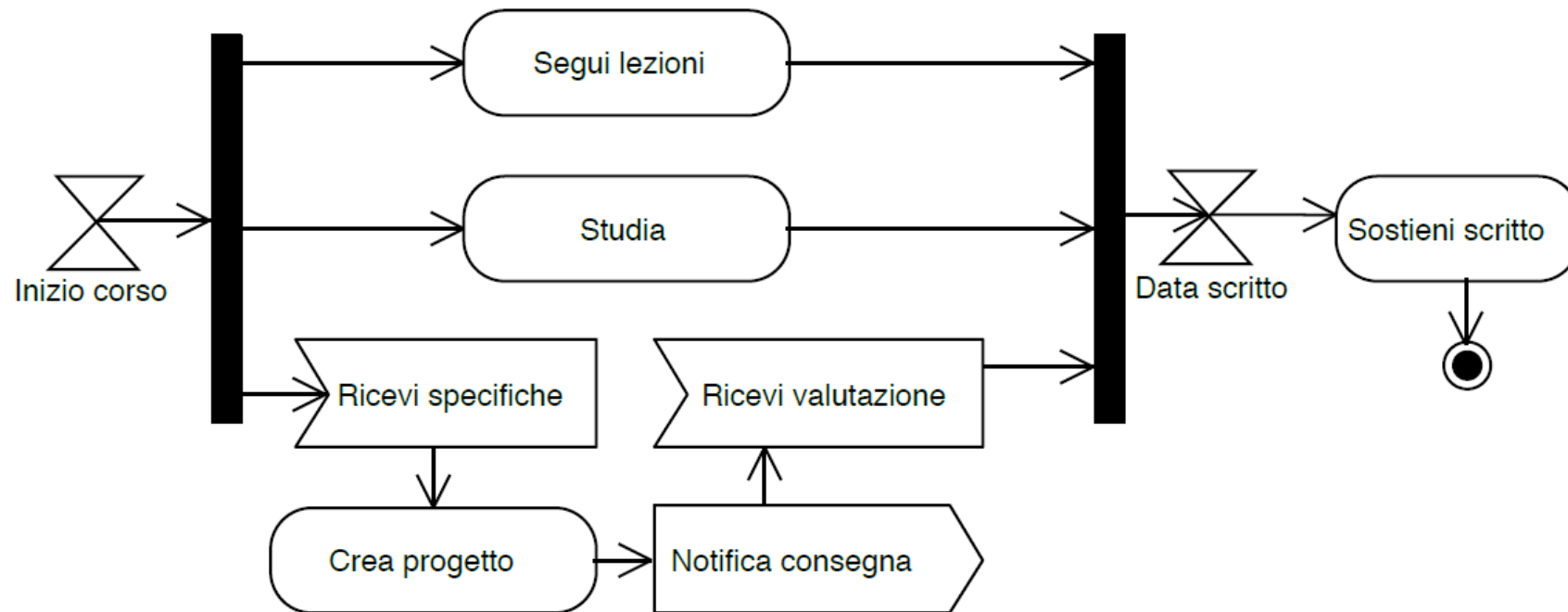
- Spesso risulta conveniente aggiungere lo stato di un oggetto per mostrarne l'evoluzione durante l'attività.
- Gli stati devono essere coerenti con la macchina a stati associata all'oggetto.
- Questo è l'anello di congiunzione tra diagrammi di attività e stato.



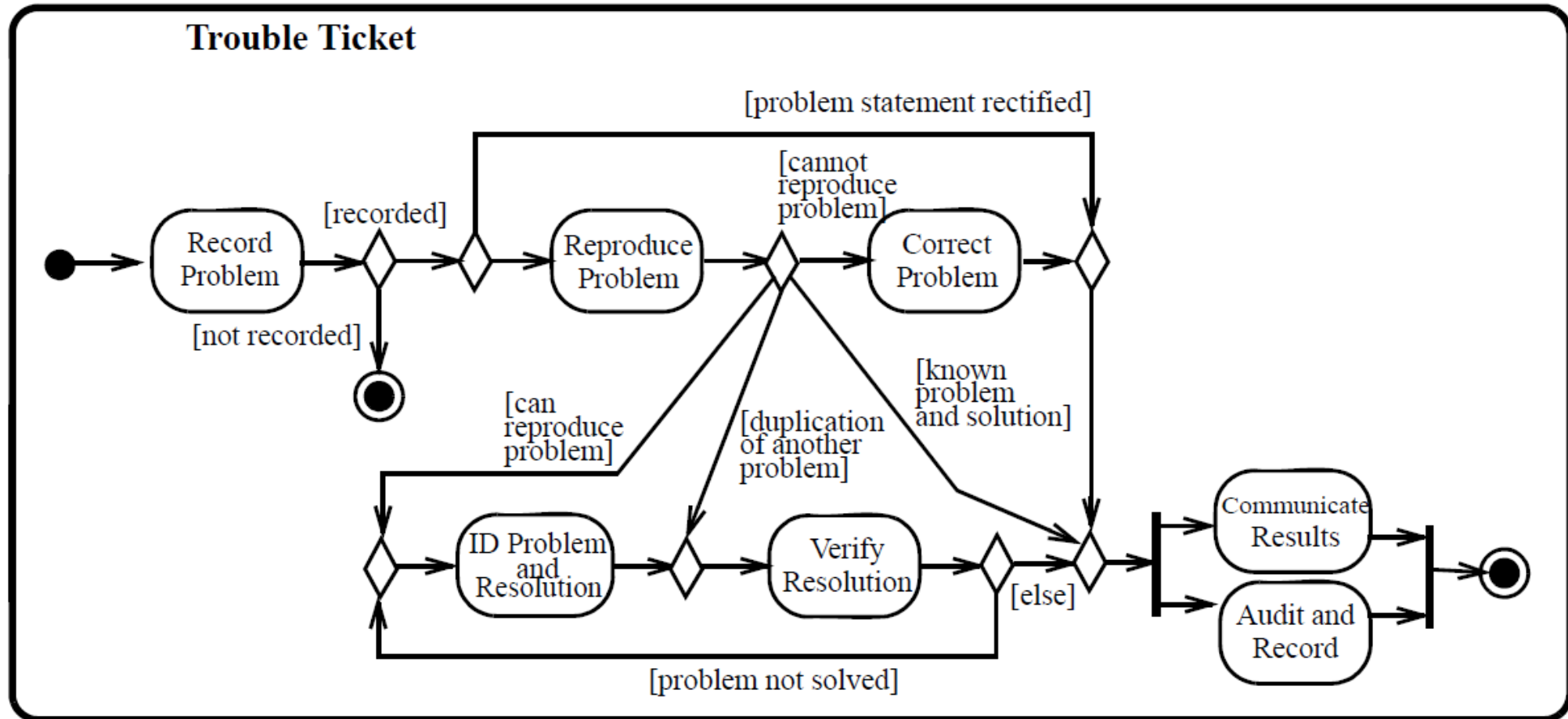
- Ci sono alcuni nodi azione specializzati che gestiscono l'invio e la ricezione di segnali.
- L'invio di segnali è asincrono e non blocca l'attività

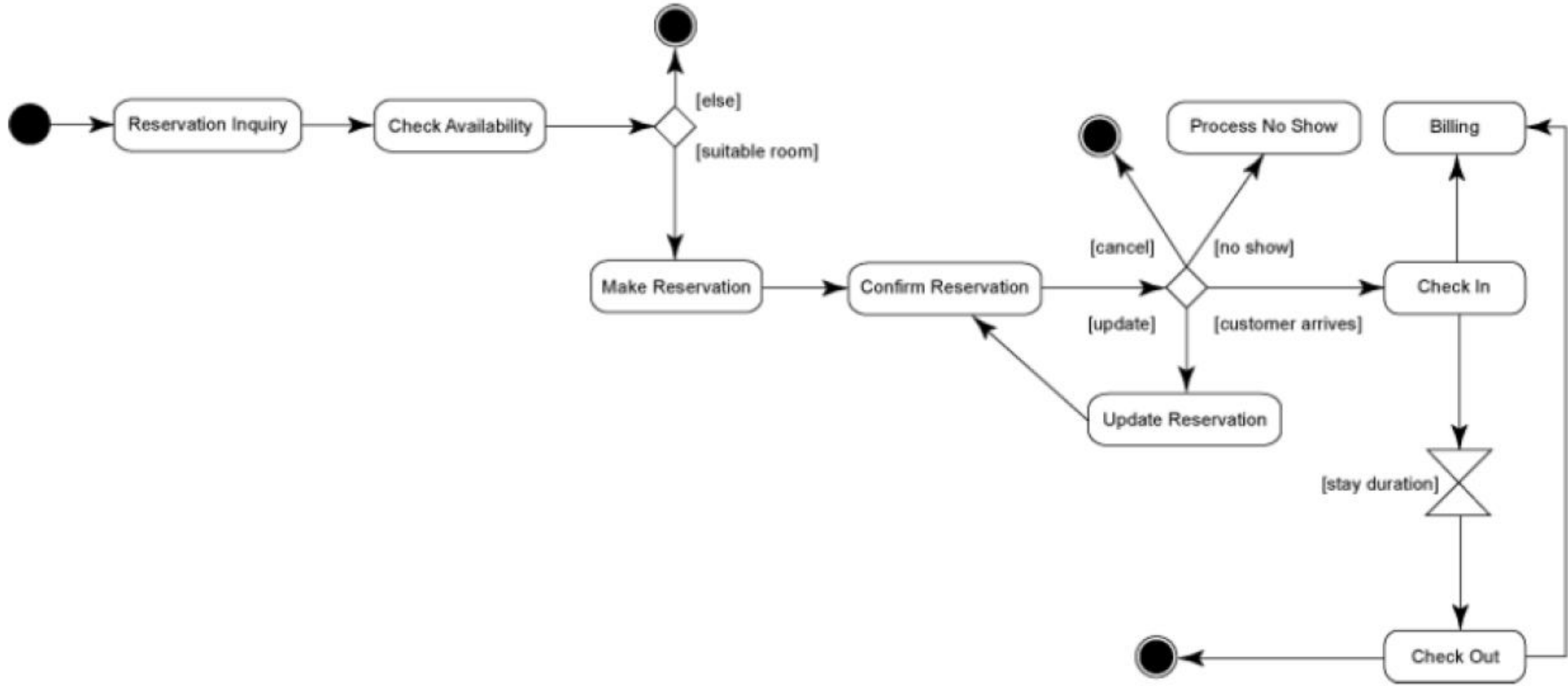


- I nodi ricezione sono attivi quando hanno token su tutti gli archi in entrata (se ne hanno) oppure durante l'intera vita dell'attività (se non ne hanno); generano token alla ricezione.
- La ricezione di eventi temporali funziona nello stesso modo, i token sono generati in base ad un'espressione temporale



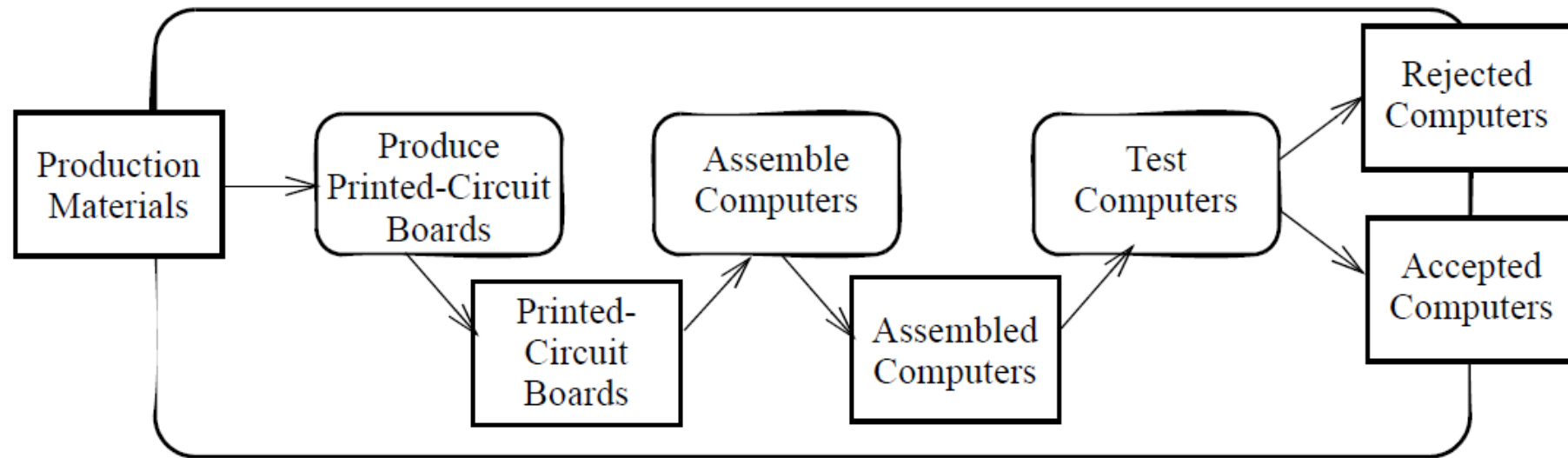
- Un'attività è costituita da un flusso di azioni che ne sono i mattoni. In effetti un'azione può invocare un'altra attività.





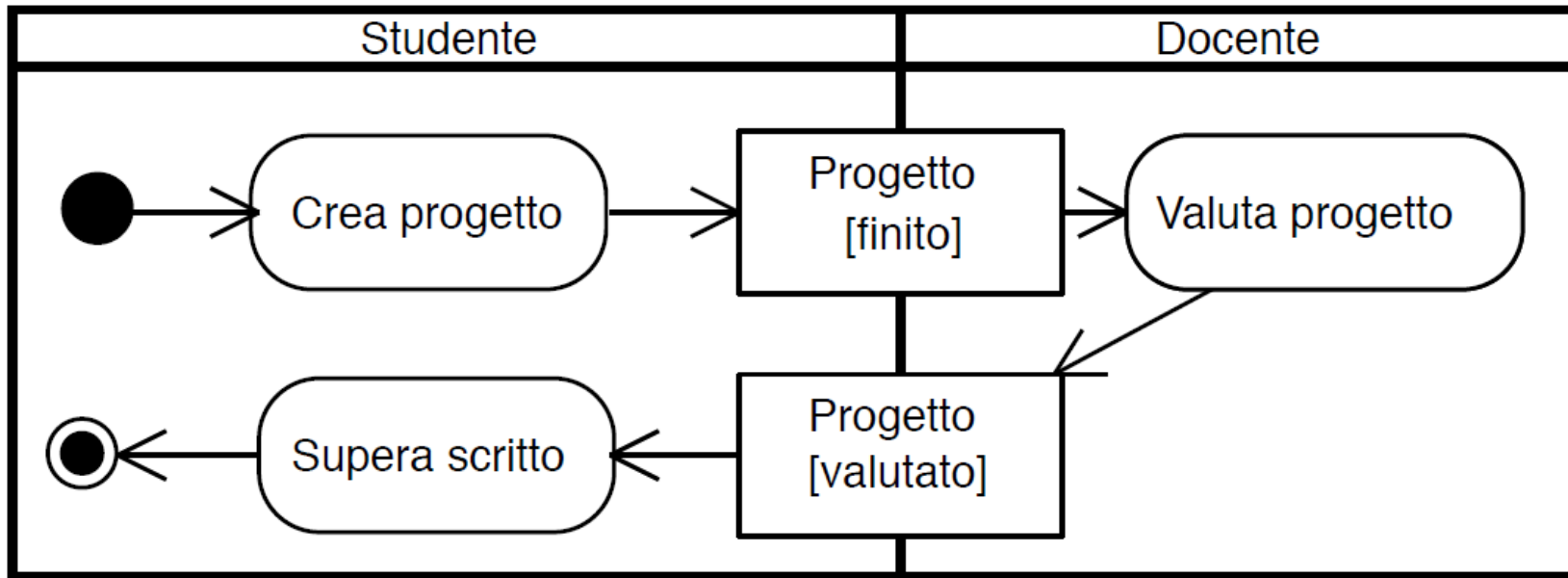
Attività: parametri e valori di ritorno

- Parametri e valori di ritorno, se esistono, si rappresentano come nodi oggetto sul bordo dell'attività.



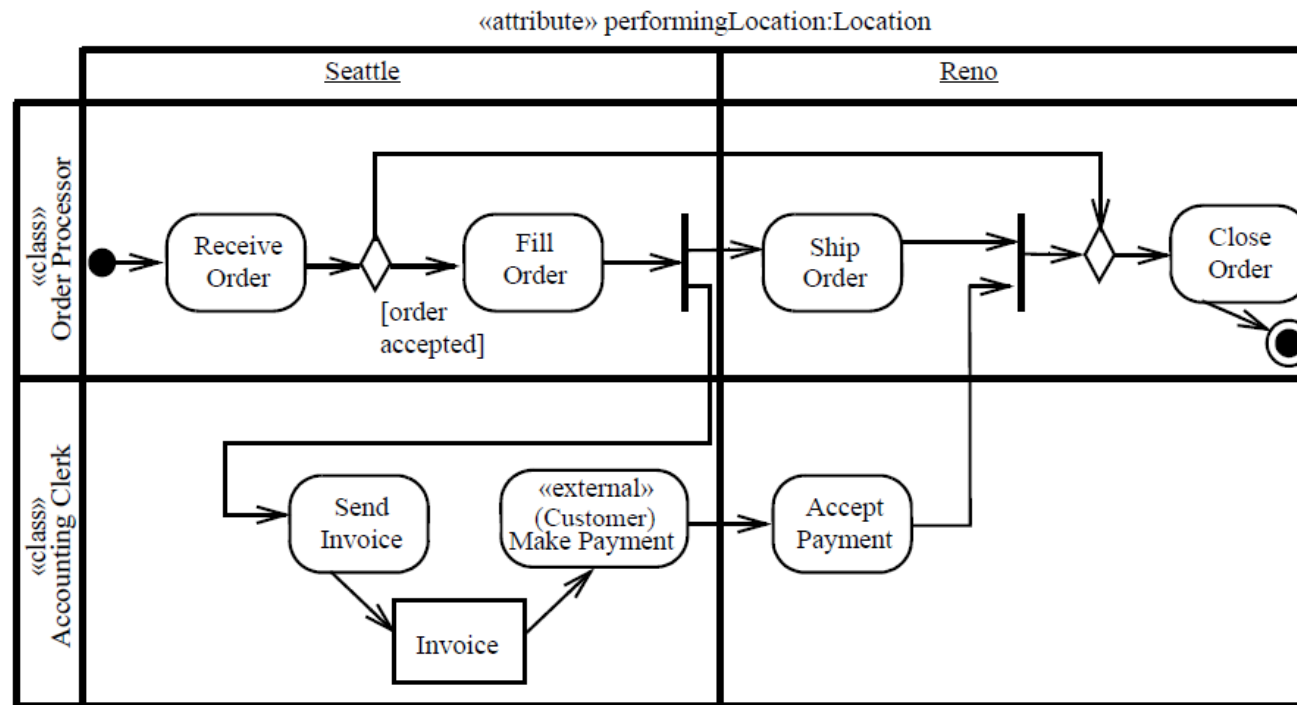
Partizioni (swimlanes)

- Suddividono il flusso dell'attività, ma non ne modificano il significato.
- La suddivisione può essere orizzontale, verticale o multidimensionale.



Partizioni (swimlanes)

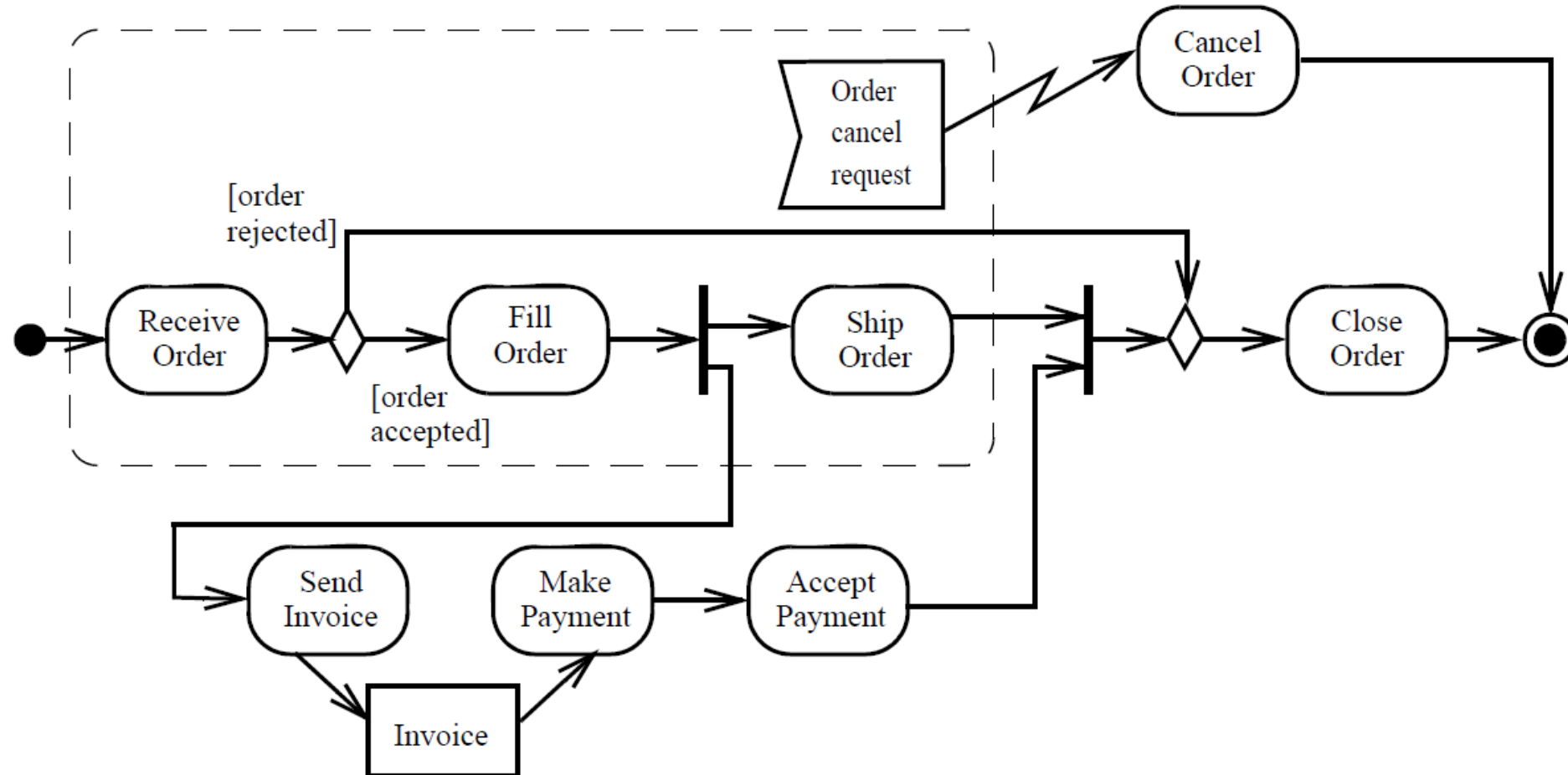
- Le partizioni sono generalmente suddivise secondo uno di questi criteri:
 - classificatori le cui istanze eseguono le varie parti
 - istanze che eseguono le varie parti
 - ruoli/parti del sistema che eseguono le varie parti
 - attributi o valori relativi alle varie parti



- Si usano per specificare una reazione che può avvenire in qualunque momento e comporta l'interruzione dell'attività.
- Esempi: eccezioni, interrupt, segnali, situazioni di errore dall'esterno.
- La notazione impiegata è quella di un'attività con i bordi tratteggiati. Uno o più archi di interrupt (a zigzag) partono da nodi interni e puntano verso nodi esterni.
- L'interrupt è generato quando un arco di interrupt è attraversato da un token: tutti gli altri token e comportamenti nella regione sono terminati.
- La ricezione di eventi all'interno della regione funziona solo se ci sono token al suo interno.

Regioni interrompibili

- L'ordine è cancellato solo se un token si trova all'interno della regione al momento della ricezione del segnale



- In UML 1.x, i due diagrammi sono in pratica la stessa cosa, ma usata in due modi diversi (i diagrammi di attività sono diagrammi di stato in cui gli stati sono azioni).
- UML 2 (qui utilizzato) ridefinisce e separa la semantica dei due diagrammi: quelli di attività si basano sulle reti di Petri, quelli di stato sulla ricerca di Harel.
- Dal punto di vista del modellatore, in UML 1.x entrambi i diagrammi sono diagrammi di stato privi di alcune funzionalità introdotte con UML 2.

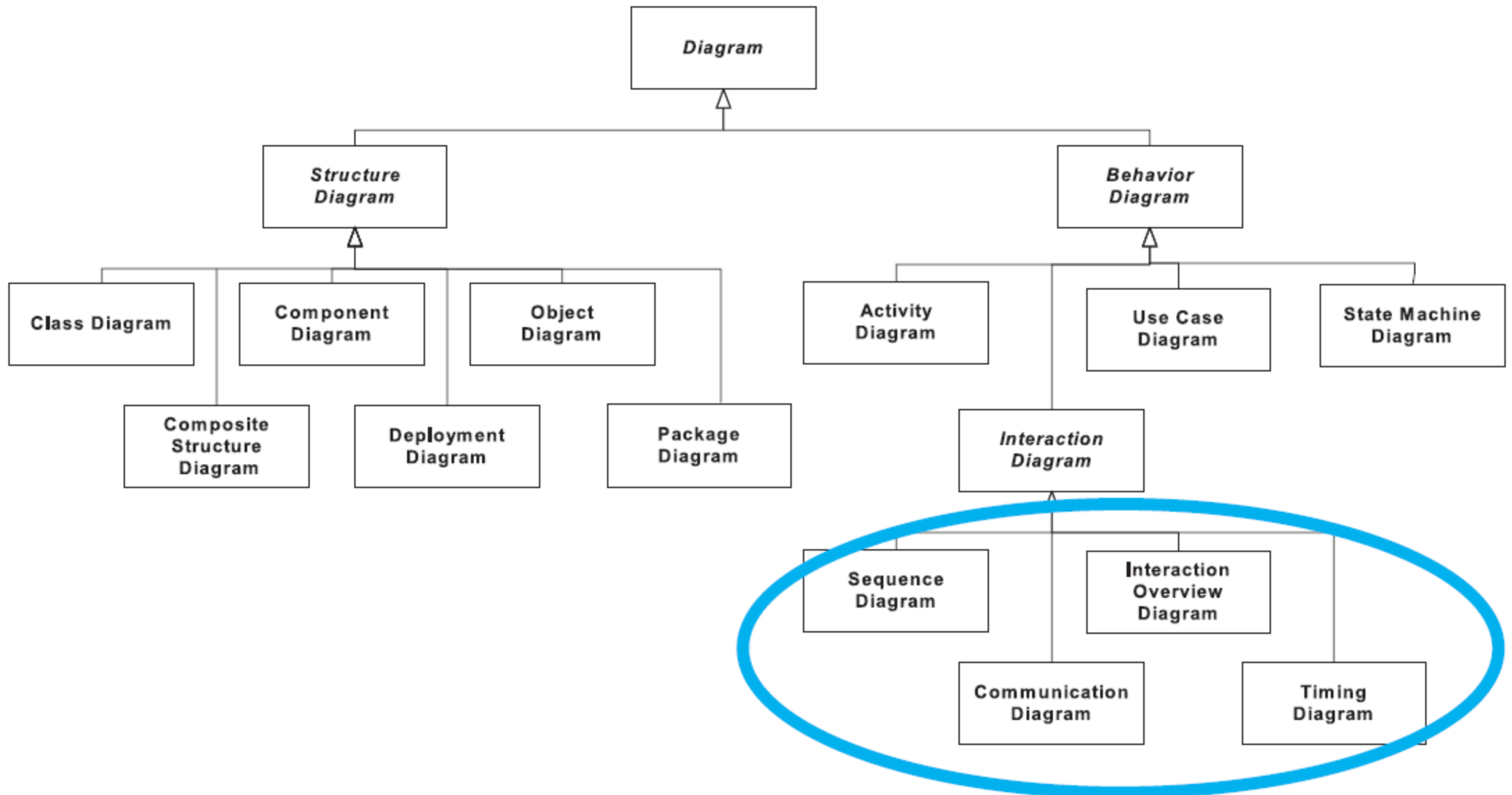
- Uno stato al contrario di un'azione dei diagrammi di attività è solitamente rappresentato con aggettivi e nomi piuttosto che verbi.
- Nei diagrammi di stato non si usano token; le transizioni sono effettuate quando avviene l'evento corrispondente.
- Lo stato iniziale e quello finale si rappresentano allo stesso modo in entrambi i diagrammi (cerchio nero e cerchio bordato).
- Altri elementi di notazione in comune con i diagrammi di attività sono i nodi decisione (decidono lo stato di destinazione in base a una guardia) e i nodi fork/join, che permettono al sistema di trovarsi in vari stati ortogonali (paralleli) allo stesso tempo.

- I diagrammi di attività descrivono un flusso di azioni che realizzano un certo comportamento specifico. L'enfasi non è sullo scambio di messaggi ma sui blocchi di comportamento.
- I diagrammi di macchina a stati si concentrano su un solo classificatore di contesto e modellano il suo stato interno in relazione al suo comportamento o alle operazioni che possono essere eseguite sulle sue istanze.
- Come tutti i diagrammi UML, possono essere usati sia a livello di analisi che di progettazione.

TOPIC 4

I diagrammi di comportamento

Tassonomia dei diagrammi UML



- Sono diagrammi di comportamento che modellano le interazioni tra varie entità di un sistema.
- Visualizzano lo scambio di messaggi tra entità nel tempo.
- Il loro scopo è mostrare come un certo comportamento viene realizzato dalla collaborazione delle entità in gioco.
- La parola chiave è realizzare: questi diagrammi mostrano la realizzazione di un comportamento offerto.
- In altri termini: il comportamento del sistema da una prospettiva interna
- Come gli altri diagrammi, possono avere diversi livelli di astrazione.

- Un diagramma di interazione necessita di due cose:
 - Un comportamento da realizzare tratto da un classificatore (classificatore di contesto), ad esempio...
 - un caso d'uso
 - un'operazione di classe
 - Una serie di elementi che realizzano il comportamento, ad esempio...
 - attori
 - istanze di classe
- Questi ingredienti provengono da diagrammi creati precedentemente.

- Ci sono 4 tipi di diagrammi di interazione, noi ci concentreremo sui primi 2.
 - Diagramma di sequenza: adatto a mostrare la sequenza temporale degli avvenimenti per ogni entità nel diagramma.
 - Diagramma di comunicazione: adatto a mostrare le relazioni strutturali e i collegamenti tra le entità e i messaggi che vi transitano.
 - Diagramma di interazione generale: adatto a mostrare la costruzione di interazioni complesse a partire da interazioni più semplici.
 - Diagramma di temporizzazione: adatto a mostrare l'evoluzione dell'interazione in tempo reale.

- Nei diagrammi di interazione generalmente non compaiono direttamente classificatori come le classi: al loro posto ci sono
 - Istanze di classificatori (oggetti, istanze di attori, etc.)
 - Linee di vita (lifeline) di classificatori (esprimono ruoli, non specifici oggetti)
- La differenza tra istanze e linee di vita è sottile, ma in generale è preferibile usare le linee di vita.
- Questi diagrammi includono anche (anzi, ne sono gli elementi principali) i messaggi, ossia i segnali che si scambiano le istanze di classificatori e/o le linee di vita

- Un'istanza:
 - rappresenta uno specifico oggetto, e solo quello
- Una linea di vita:
 - è più generale
 - rappresenta un'istanza arbitraria di un classificatore
 - fornisce modi per specificare come quest'istanza viene scelta
 - esprime il ruolo giocato da un'istanza senza preoccuparsi della sua identità

- Entrambe usano la notazione del loro classificatore, ma le linee di vita non sono sottolineate.
- La notazione completa per una linea di vita è: nome [selettore] : tipo
- Il selettore è un'espressione booleana opzionale che permette di scegliere un particolare rappresentante del classificatore, ad es. [id="1234"].
- Senza selettore la linea di vita rappresenta un'arbitraria istanza.

Jim : Person

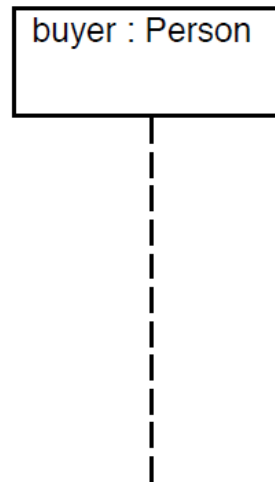
buyer : Person

TOPIC 5

I diagrammi di sequenza

I diagrammi di sequenza

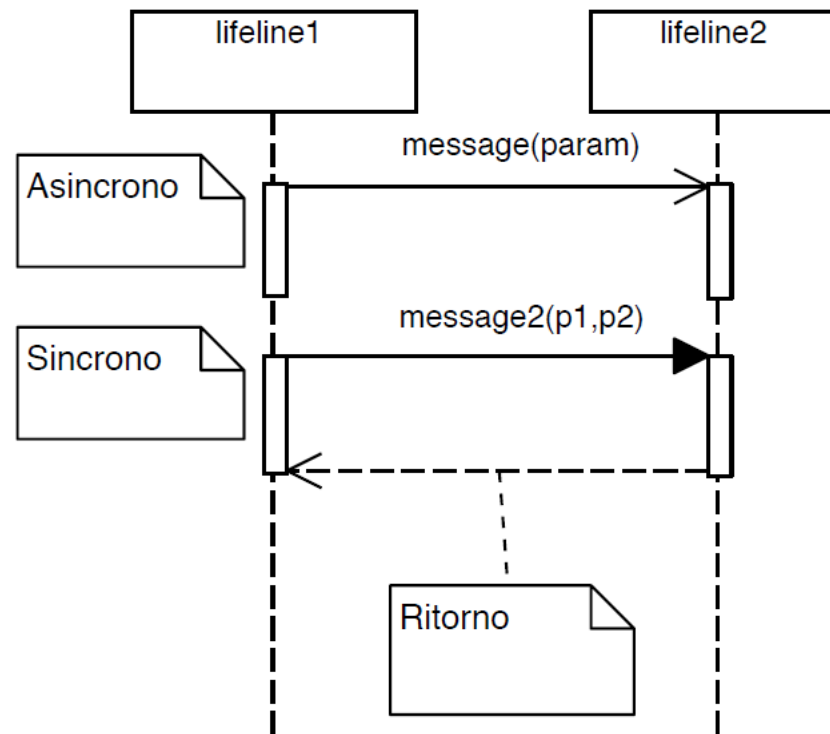
- Evidenziano l'ordine temporale delle invocazioni dei metodi
- Una linea verticale tratteggiata indica una sequenza temporale.
- Gli eventi hanno luogo nel loro ordine sulla linea; le distanze sono irrilevanti.
- Più linee di vita interagiscono per realizzare il comportamento offerto, scambiandosi messaggi



- Rappresentano comunicazioni tra linee di vita: segnali, chiamate di operazioni, creazione e distruzione di oggetti.
- Sette tipi definiti:
 - chiamata sincrona
 - chiamata asincrona
 - ritorno da chiamata
 - creazione
 - distruzione
 - messaggio trovato
 - messaggio perso

Tipi di messaggi: chiamata

- Comunicazione sincrona vs. comunicazione asincrona.
- I rettangoli di attivazione (indicano un focus di controllo) sono opzionali.

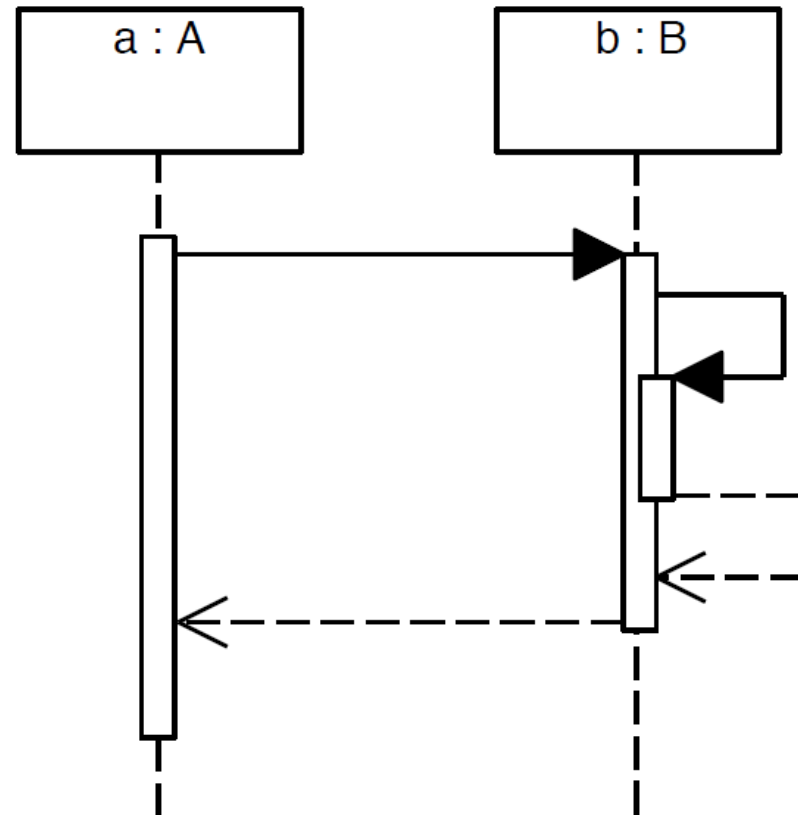


- I messaggi sincroni fanno bloccare chi li manda fino al messaggio di ritorno del destinatario.
- Nei messaggi asincroni il mittente non si aspetta un messaggio di ritorno e prosegue immediatamente.
- I messaggi di ritorno sono opzionali e in generale inclusi solo quando non ostacolano la leggibilità del diagramma oppure per segnalare valori di ritorno.
- La distinzione tra messaggi sincroni e asincroni in genere emerge in fase di progettazione.
- In fase di analisi, conviene indicare tutti i messaggi come sincroni (è il caso più vincolante).

- In fase di analisi generalmente basta includere il nome del messaggio.
- Se si desidera maggiore dettaglio, si include una lista di parametri tra parentesi.
- Si possono anche usare guardie per verificare condizioni
- Si possono indicare i parametri formali, quelli attuali, o entrambi (in quest'ordine: `getArea(shape=g)`).

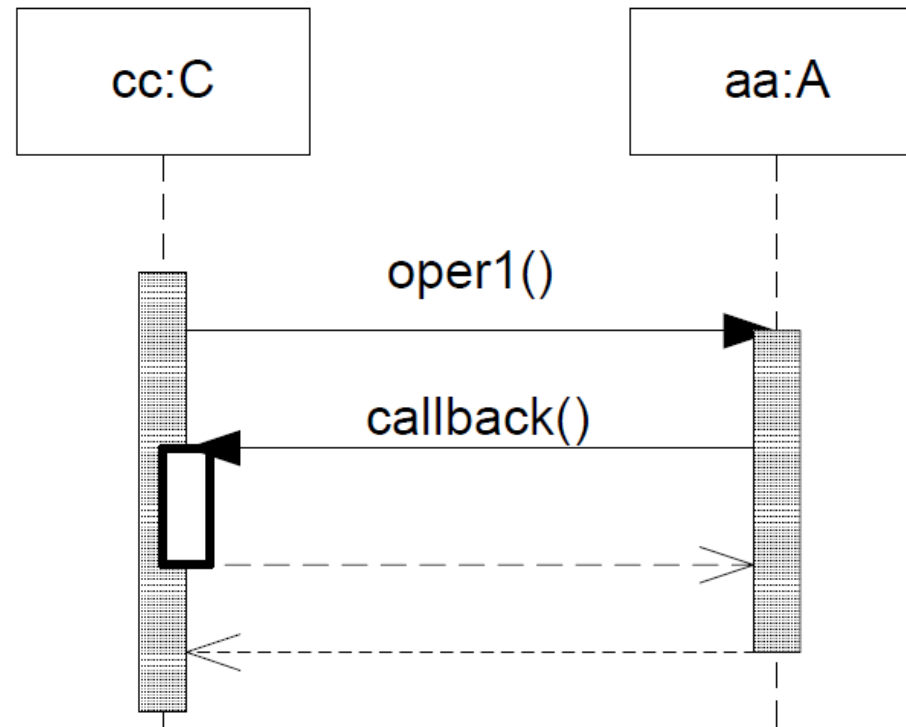
Attivazioni annidate

- Le linee di vita possono mandare messaggi a se stesse.
- Si tratta di un caso frequente (es. invocazione di operazioni private).



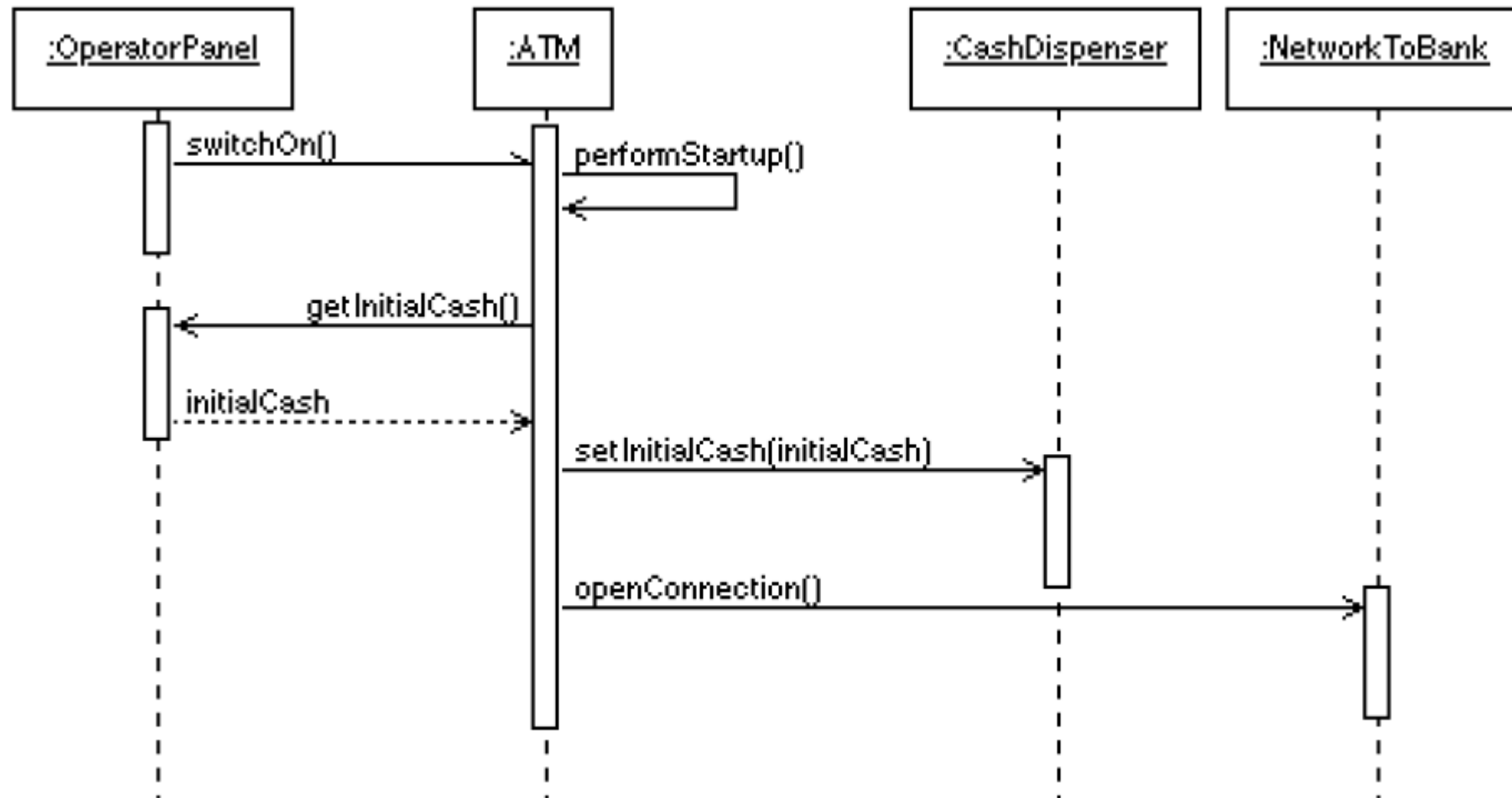
Attivazioni annidate

- Le attivazioni possono generare altre attivazioni sulla linea di vita chiamante (es. callback).

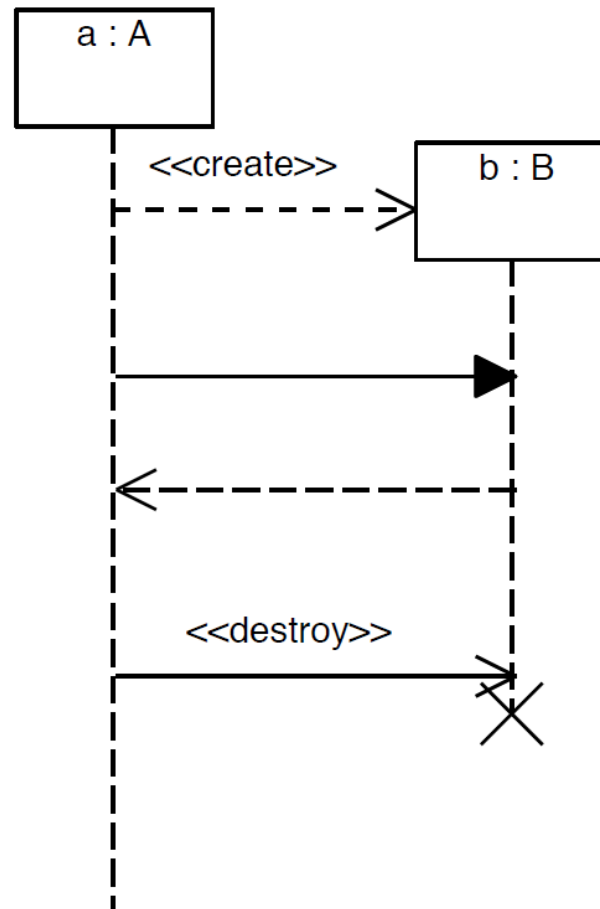


Esempio completo: ATM machine

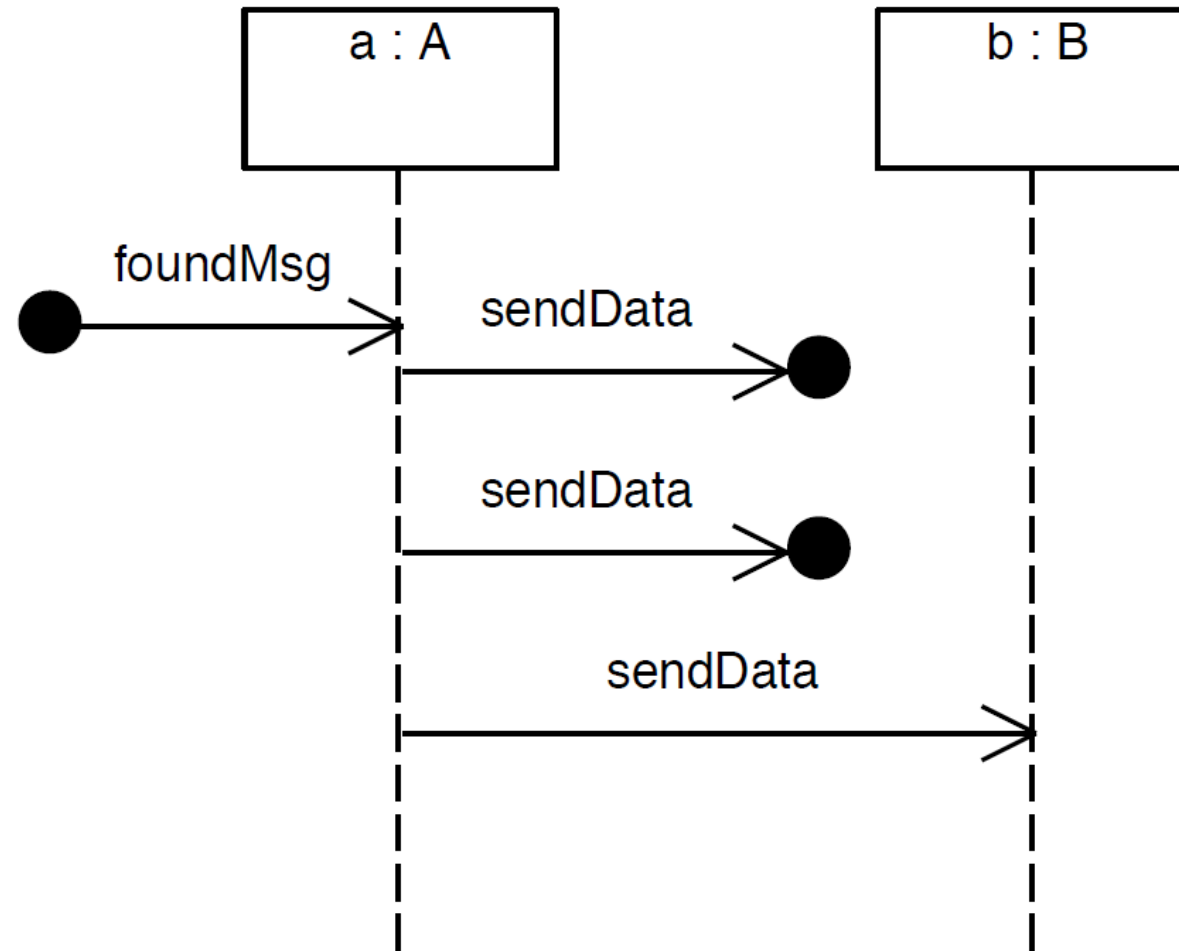
- Le attivazioni possono generare altre attivazioni sulla linea di vita chiamante (es. callback).



- Le attivazioni possono generare altre attivazioni sulla linea di vita chiamante (es. callback).



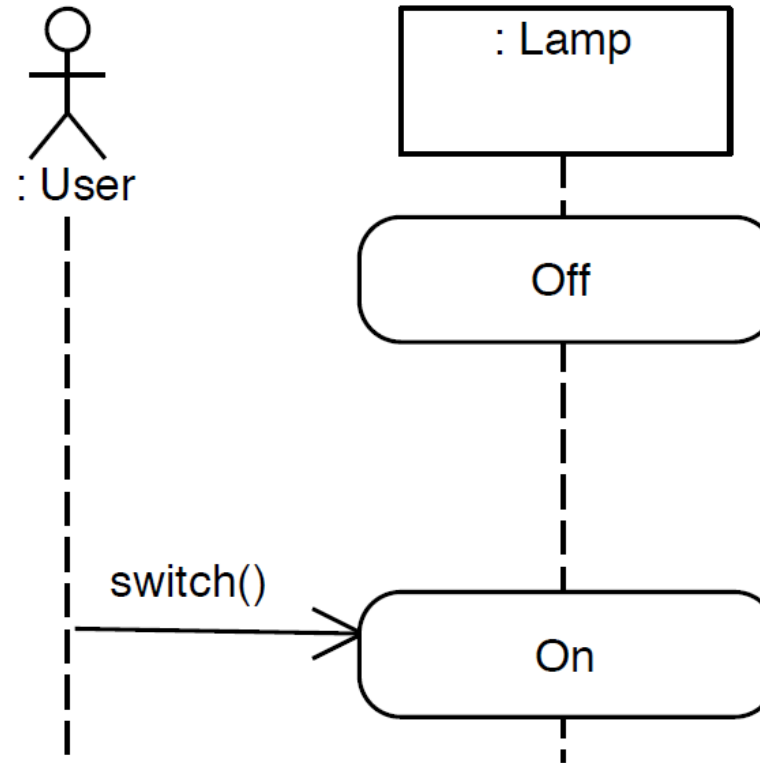
- Nel messaggio di creazione lo stereotipo può essere seguito dal nome di un'operazione e relativi parametri (costruttore).
- Questo si rende necessario nel caso si lavori con linguaggi di programmazione senza costruttori espliciti.
- Differenti linguaggi di programmazione OO gestiscono la distruzione in modi diversi (esplicita, garbage collector).
- In fase di analisi, basta immaginare che dopo la distruzione, l'oggetto non è più accessibile



- Raramente usati in fase di analisi.
- I messaggi trovati indicano situazioni in cui il mittente è sconosciuto al momento della ricezione, proviene dall'esterno del diagramma o i dettagli della provenienza non interessano.
- I messaggi persi permettono di visualizzare il comportamento nel caso un messaggio non giunga a destinazione (situazione di errore).

- Uno stato è definito come una condizione o situazione durante la vita di un oggetto in cui esso soddisfa una condizione, esegue un'attività o aspetta un evento.
- Ogni oggetto in UML può avere una macchina a stati associata che ne descrive gli stati possibili.
- I diagrammi di stato (state machine diagram) sono i più adatti a rappresentare gli stati e loro transizioni, ma può essere conveniente mostrare alcuni cambiamenti di stato nei diagrammi di sequenza.
- Generalmente, ci si limita agli stati principali all'oggetto, mettendo in risalto i messaggi che provocano un cambiamento di stato.

- In UML, gli stati si rappresentano con rettangoli arrotondati. I cambiamenti di stato sono generalmente la conseguenza di uno o più messaggi.

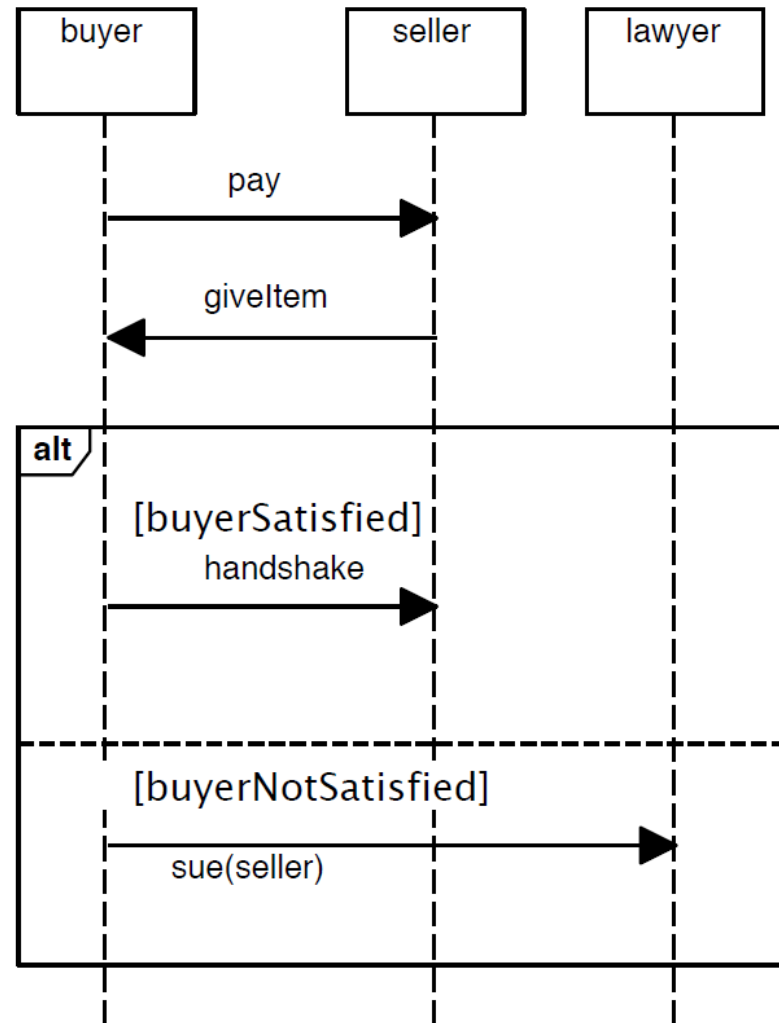


- Le primitive introdotte fin qui permettono di rappresentare sequenze di eventi senza ramificazioni; tuttavia, per realizzare i casi d'uso è necessario poter descrivere sequenze più complesse.
- Le sequenze degli eventi in un caso d'uso contengono parole chiave come if, then, else, for e while.
- I diagrammi di sequenza permettono di tradurre questi costrutti ed inserirli nel diagramma.
- Si ricorre ai frammenti combinati

- Un frammento combinato è una sottoarea di un diagramma di sequenza che racchiude una parte dell'interazione e le modalità della sua esecuzione.
- Gli elementi di un frammento combinato sono:
 - un operatore, che specifica come il frammento viene eseguito
 - uno o più operandi, sottoaree del diagramma di sequenza che possono essere eseguite
 - zero o più condizioni di guardia, espressioni che determinano quali operandi sono eseguiti.

- La sintassi UML per un frammento combinato è la seguente:
 - un rettangolo con il nome dell'operatore in alto a sinistra racchiuso nel simbolo di diagramma (rettangolo con angolo «tagliato»)
 - gli operandi seguono come strisce orizzontali in sequenza, separati da linee tratteggiate
 - la guardia, se presente, compare dopo l'operatore oppure nella parte alta di un operando tra parentesi quadre
- Il rettangolo del frammento combinato si estende orizzontalmente per includere le linee di vita interessate dall'interazione.
- La guardia può includere qualunque tipo di condizione booleana, incluso OCL.

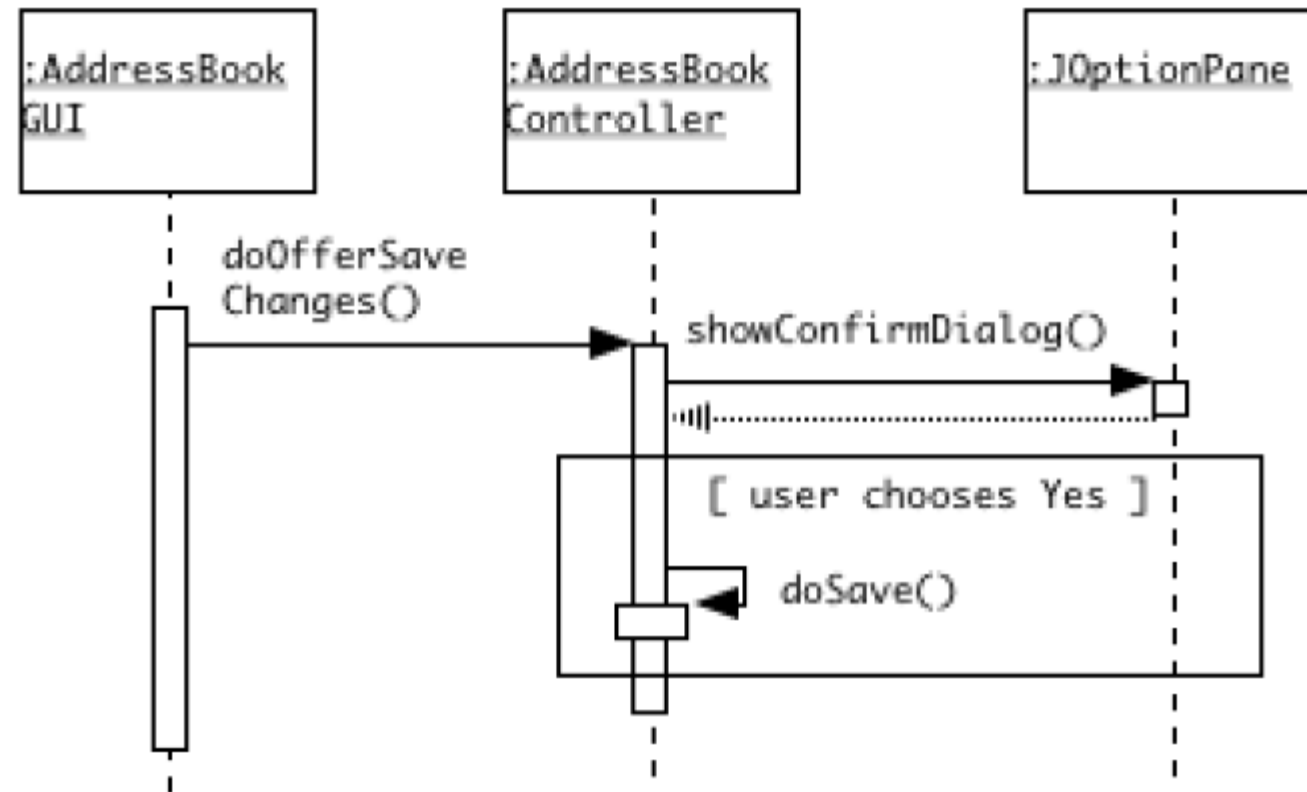
Frammenti combinati: esempio



- L'operatore determina la semantica dell'esecuzione del frammento.
- UML specifica parecchi tipi di operatori, ma solo alcuni sono usati frequentemente (quelli che corrispondono alle primitive di controllo del flusso definite dai linguaggi di programmazione).
- I più usati:
 - opt corrisponde a if/then
 - alt corrisponde a case/select
 - loop corrisponde a for/while
 - break corrisponde a break

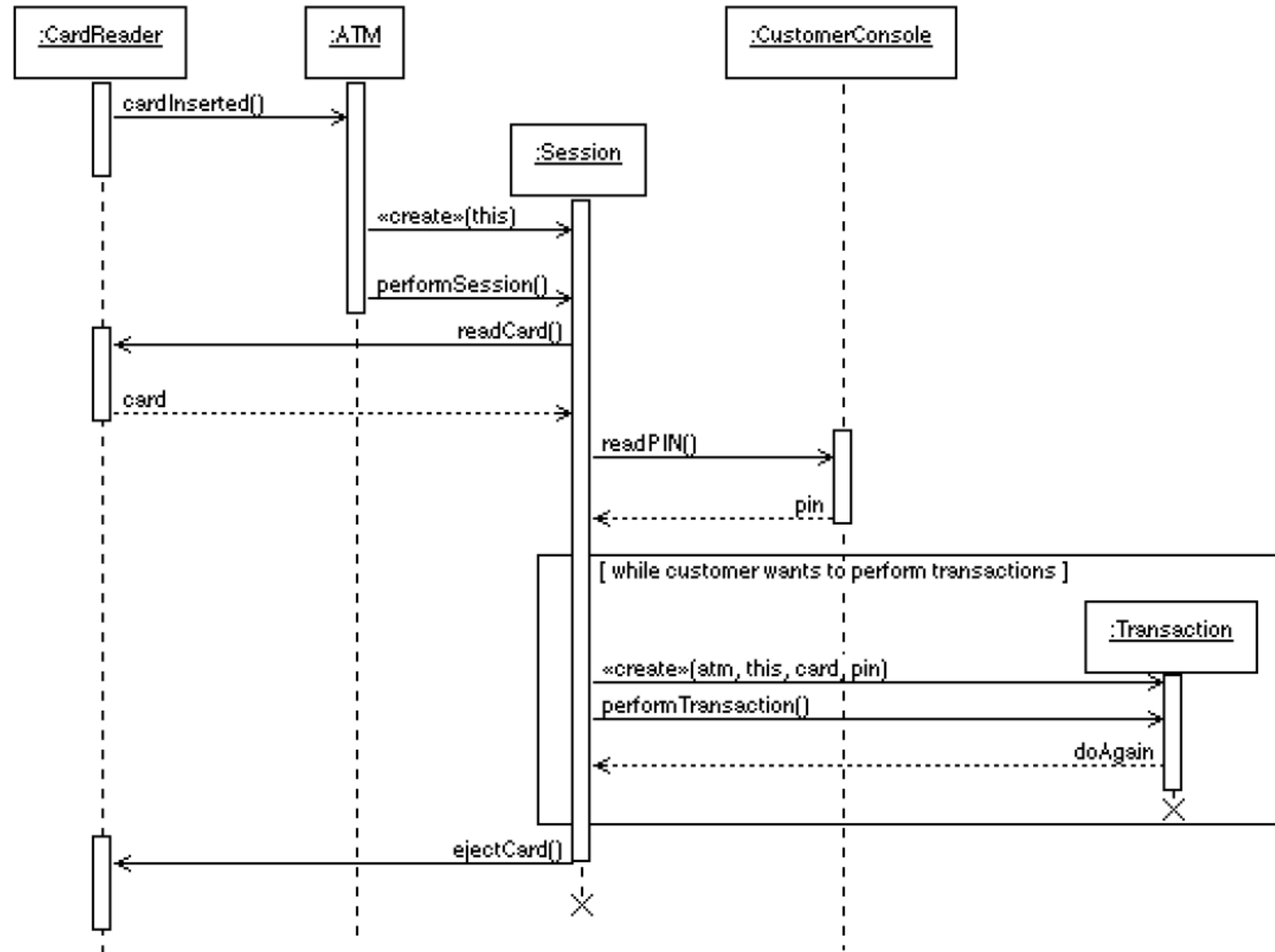
- opt accetta solo un operando, che viene eseguito se e solo se la guardia è valutata true.
- alt accetta un qualunque numero di operandi e ne esegue al più uno.
 - esamina la lista a partire dal primo operando, valuta le guardie ed esegue solo il primo operando la cui guardia è vera
 - l'operando opzionale [else] è eseguito se nessuna delle guardie è vera
 - le condizioni di guardia devono essere mutualmente esclusive; al massimo una può essere vera nello stesso istante, altrimenti il modello non è corretto

- L'operatore opt può anche essere omesso per questioni di leggibilità



- Un solo operando, sintassi speciale: loop min,max [condizione]
- Esegue l'operando min volte, e poi al massimo altre (max - min) volte mentre la condizione è vera.
- Si possono omettere min e max.
- La condizione è molto flessibile (può essere espressa in linguaggio naturale), e permette di ciclare su un insieme di oggetti (es. condizione [forEach oggetto in lista]).

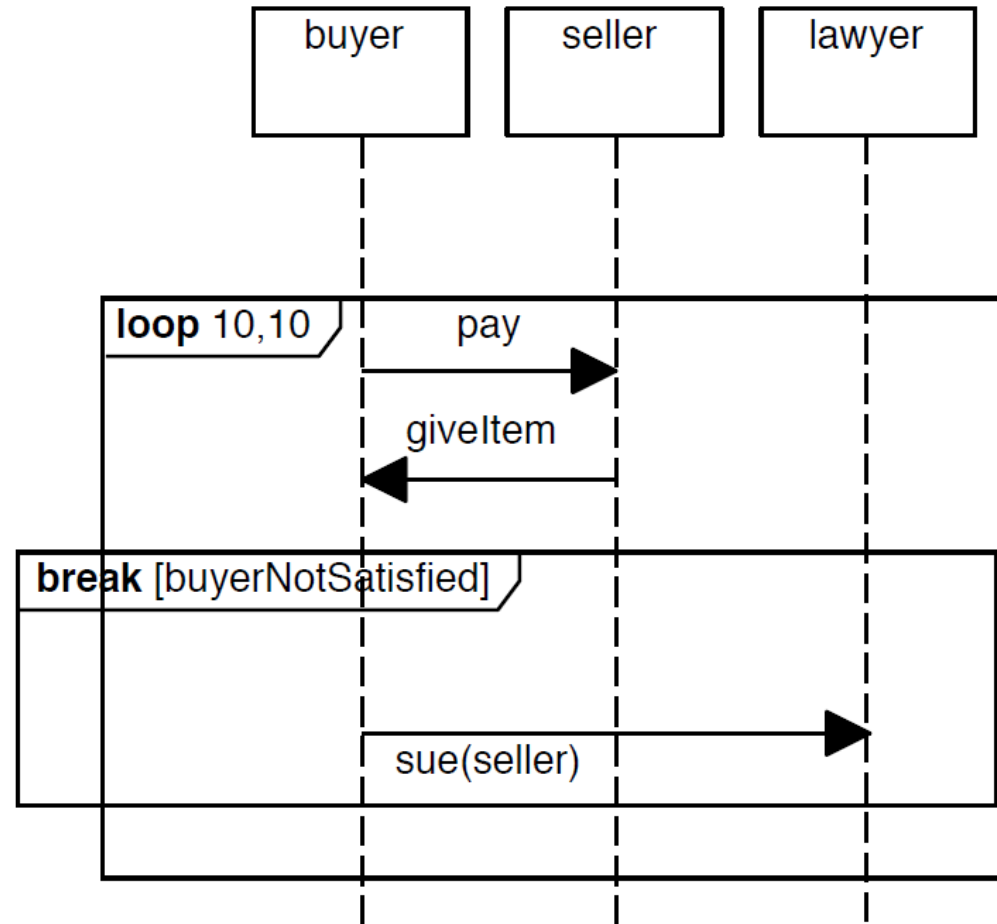
Esempio loop



- Operando singolo, si usa per terminare l'esecuzione del frammento di interazione corrente (spesso si tratta di un operatore loop).
- Se break ha una condizione di guardia ed è vera, viene eseguito l'operando ma non il resto dell'interazione dopo il frammento break.
- Se la condizione di guardia esiste ed è falsa, non viene eseguito l'operando e si continua normalmente.
- Se non c'è una guardia, la scelta tra continuare e saltare è non-deterministica.
- Il frammento break deve essere globale rispetto a quello che interrompe: nella notazione UML, lo interseca ma si trova ad un livello di nesting superiore.

Esempio: loop e break

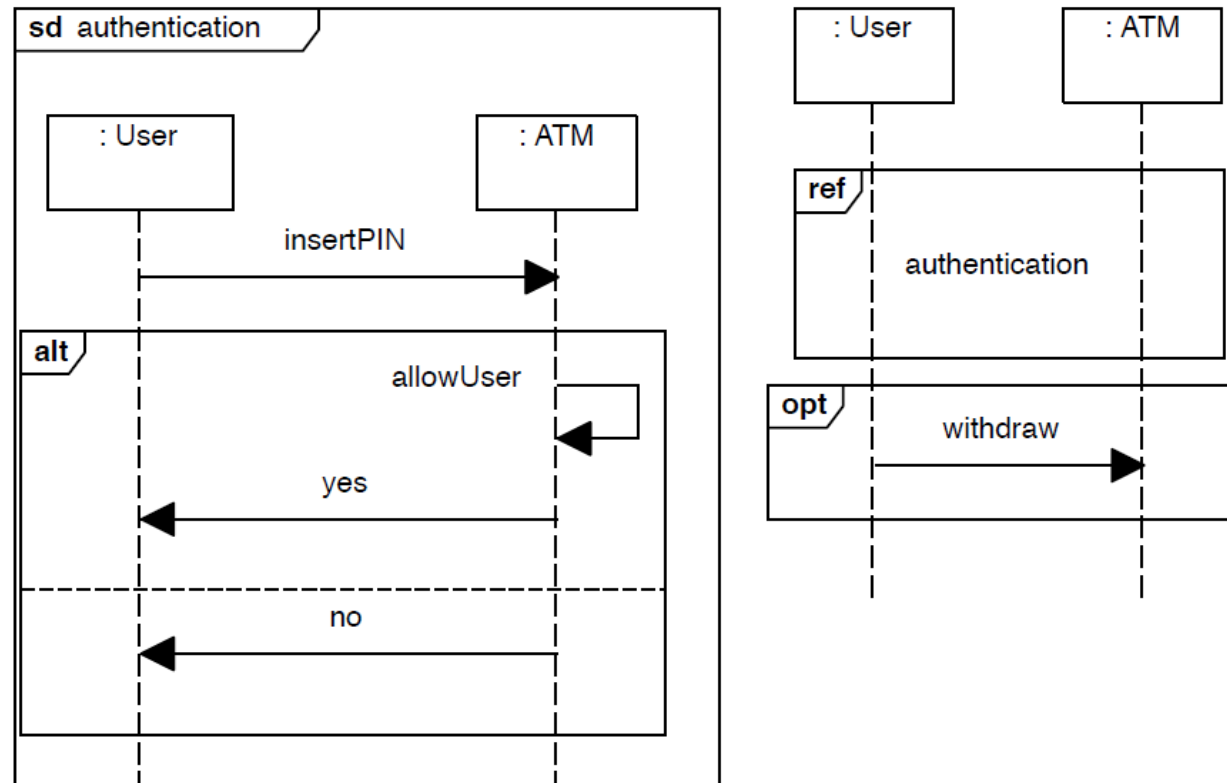
- Vengono completate al massimo 10 transazioni, se si passa a vie legali le transazioni finiscono.



- Per rendere i diagrammi di sequenza modulari:
 - ref: l'operando contiene il nome di un'altra interazione che viene eseguita qui.
- Altri molto meno usati:
 - par: gli operandi sono eseguiti in parallelo, senza vincoli sull'ordinamento dei messaggi appartenenti a operandi diversi.
 - critical: sezione critica, l'operando è eseguito in maniera atomica e senza interruzioni dalle linee di vita interessate, anche se all'interno di par o altri frammenti.
 - neg: l'operando rappresenta un'interazione che non deve accadere.
 - assert: l'operando rappresenta l'unica interazione ammissibile in quel momento.

Usare ref

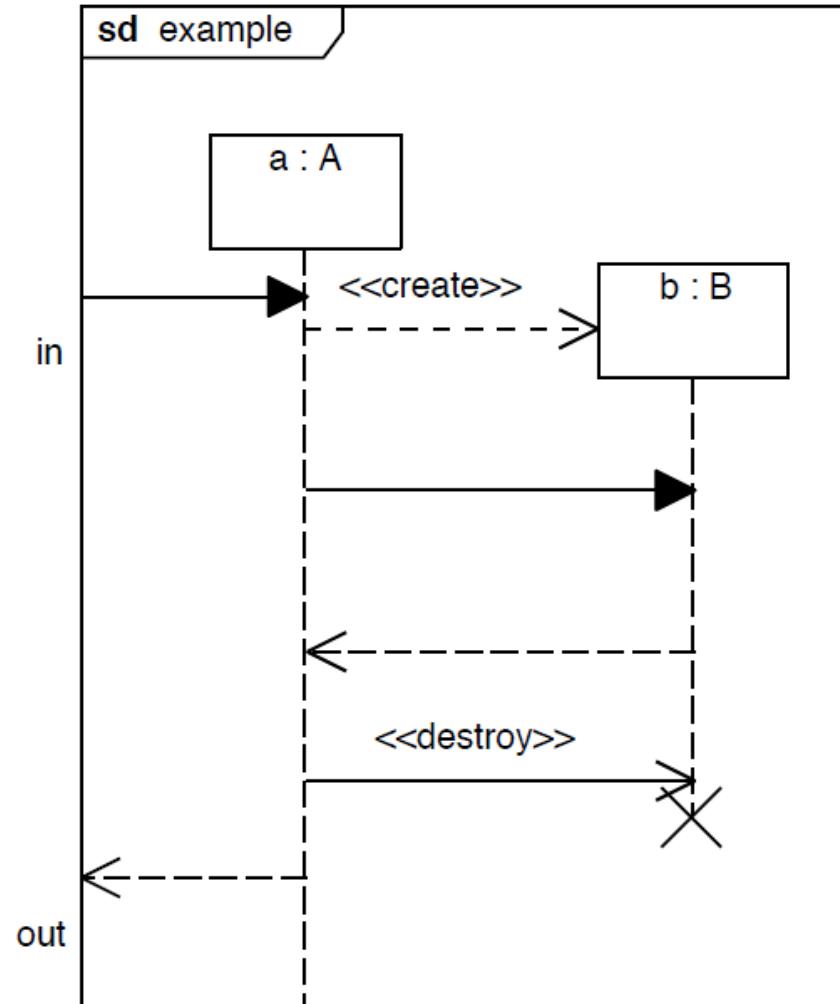
- A destra, l'interazione di sinistra inclusa in un'altra interazione. Si tratta del modo più semplice di usare ref.
- sd vuol dire «sequence diagram», ma vale per tutti i diagrammi di interazione.



- Si tratta di un qualunque punto sulla cornice esterna del diagramma, che può essere la fonte o la destinazione di una freccia.
- Usati per modellare la comunicazione delle linee di vita del diagramma con l'esterno.
- I gate possono avere un nome e comparire qualunque numero di volte nello stesso diagramma o in diagrammi diversi.
- Possono essere usati in combinazione con ref, forniscono l'interfaccia di collegamento tra l'interazione chiamante e quella chiamata (il frammento combinato può avere gli stessi gate a cui agganciare messaggi).

Gate: esempio

- Vengono definiti due gate, in e out



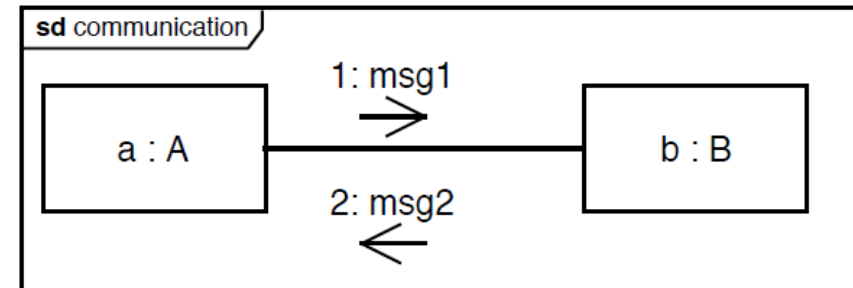
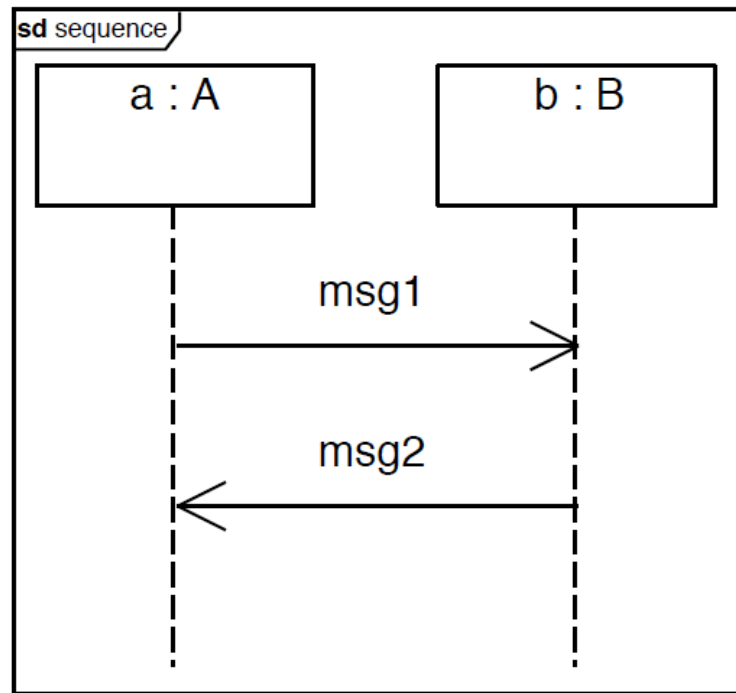
TOPIC 6

Diagrammi di comunicazione

- Chiamati diagrammi di collaborazione in UML 1.x.
- In UML 1.x erano semanticamente equivalenti ai diagrammi di sequenza, in UML 2 non sono altrettanto espressivi.
- Si tratta comunque di diagrammi utili a visualizzare i canali di comunicazione tra le entità.
- Sono simili ai diagrammi degli oggetti, ma i link tra gli oggetti trasportano messaggi come nei diagrammi di sequenza.

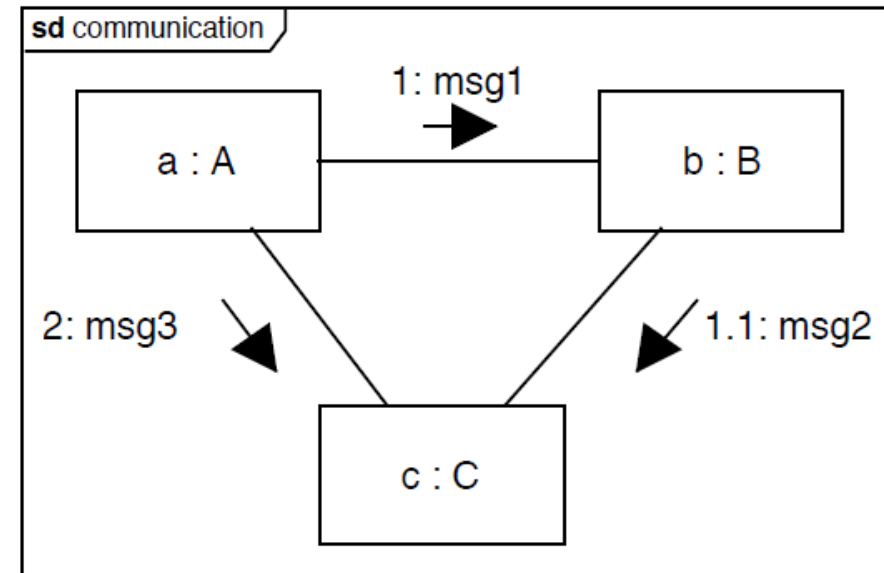
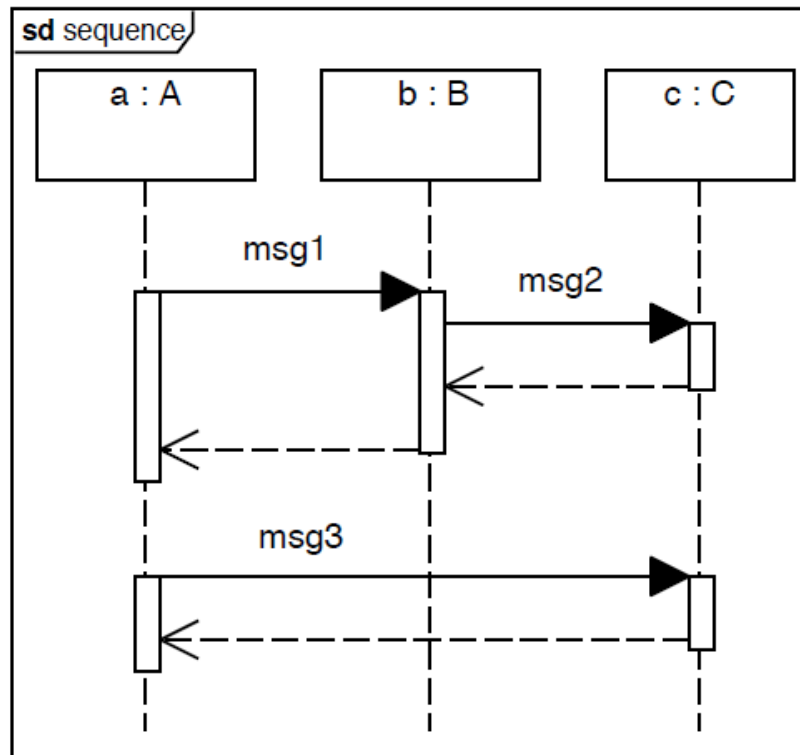
Sequenza vs. comunicazione

- Questi diagrammi sono equivalenti.
- I messaggi e le loro frecce seguono le stesse regole nei diagrammi di comunicazione



Numerazione messaggi

- Nei diagrammi di comunicazione è necessario numerare i messaggi.
- Poiché msg2 si trova nel focus di controllo di msg1 (viene generato come parte dell'esecuzione di msg1), il suo numero di sequenza è quello di msg1 seguito da '.' e un numero di sottosequenza. Ingegneria del Software () I diagrammi di inter



- Il numero di sequenza ha tanti componenti quanti sono i livelli di profondità delle attivazioni: es. 2.5.1.10 .
- I numeri di sequenza possono velocemente diventare complessi e danneggiare la leggibilità del diagramma.
- Per questo motivo, da un lato si cerca di limitare la complessità dei diagrammi di comunicazione...
- ... e dall'altro è consentito raggruppare vari messaggi correlati in uno solo per ridurre il numero (specialmente in fase di analisi).

- Si può rappresentare una condizione if/then nei diagrammi di comunicazione.
- Il numero di sequenza è seguito da una guardia tra parentesi quadre, ad esempio

1.2.1 [!error]: msg(param1,param2)

- Il messaggio è inviato solo se la guardia è vera.
- Questo costrutto può complicare il diagramma molto in fretta, e dovrebbe essere usato solo in casi semplici.

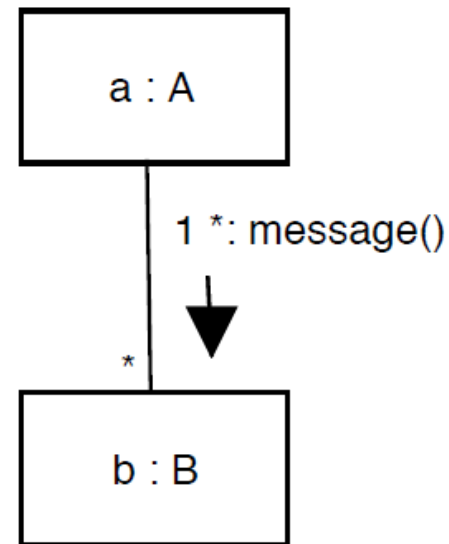
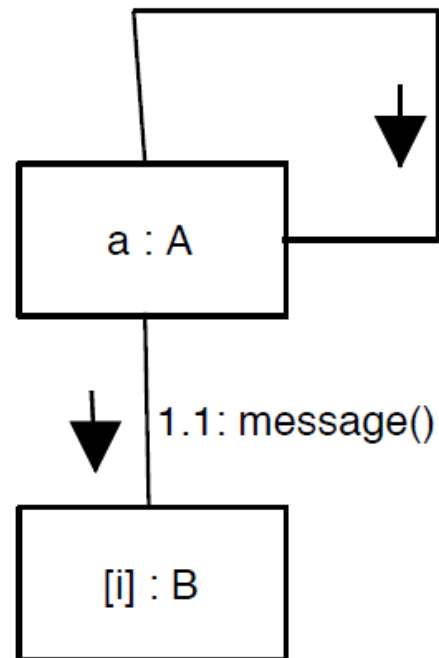
- L'operatore di iterazione nei diagrammi di comunicazione è *. Per esempio

1.2.1 * [for i=1 to n]: msg(param1,param2)

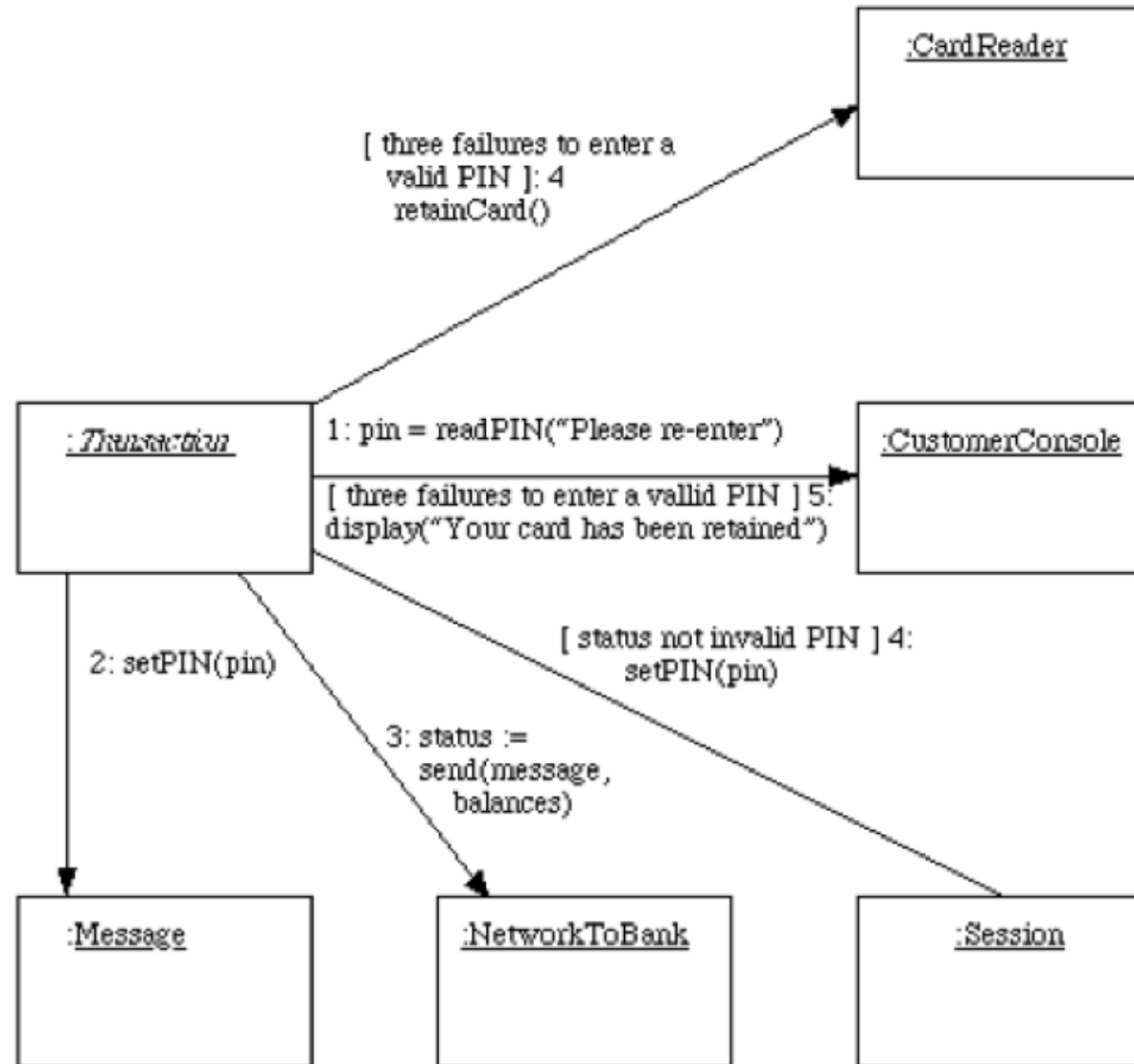
- Si può sostituire * con * || che significa invio parallelo dei messaggi.
- La guardia può essere scritta in qualunque linguaggio o pseudocodice, compresa la notazione [loop min, max [condizione]] dei diagrammi di sequenza.
- Come mandare lo stesso messaggio ad una collezione di istanze con l'iterazione?
- Due modi: con indici o con molteplicità.

- Nel primo caso [i] è una sola istanza, rappresentata tramite un indice.
- Nel secondo caso a è collegato a molti b, e l'operatore * manda il messaggio a tutti.

1 * [for i=1 to n]: sendMessageTo(i)



Esempio completo: ATM Machine (Pin Error)



Conclusioni sui diagrammi di interazione

- Si usano i diagrammi di interazione per mostrare come varie entità collaborino nel fornire un comportamento desiderato, ad esempio un caso d'uso.
- Possono essere usati sia in fase di analisi che in fase di progettazione.
- Aiutano a scoprire e correggere inconsistenze nel comportamento desiderato (ad esempio una specifica scorretta del caso d'uso).
- Spesso è utile modellare una situazione semplificata con un diagramma di comunicazione, per poi passare a uno di sequenza per i dettagli.

TOPIC 7

Esempi Completi

- <http://www.math-cs.gordon.edu/courses/cs211/AddressBookExample/>
- <http://www.math-cs.gordon.edu/courses/cs211/ATMExample/>